CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili





UNIVERSITY OF BEDFORDSHIRE (UOB)

DESKTOP APPLICATIONS DEVELOPMENT AND SOFTWARE ENGINEERING

Unit Code: CIS116-2

Assignment 1 – Control an Elevator - A C# Project – Test Report

(Two-Lift Controller in C#)

Submitted By:	University ID	Submission Date
Pujan Rasaili	2528898	November 03, 2026

Table of Contents

1. Introduction 4
2. Problem Statement 4
2.1 Major Challenges 4
3. Proposed Solution 4
2.1 Major Challenges 4
4. Aims and Objectives 5
4.1 Aim 5
4.2 Objectives 5
5. Planning 5
5.1 Weekly Schedule 5
5.2 Functional Requirements 5
5.3 Non-Functional Requirements 6
6. System/Technical Requirements 6
6.1 Hardware Requirements 6
6.2 Software Requirements 6
7. System Design 7
7.1 Use Case Diagram 7
7.2 Class Diagram 7
7.3 ER Diagram 8
7.4 Activity Diagram 9
8. Tech Stack Implementation 12
8.1 Language Used 12
8.2 Object-Oriented Programming Implementation 12
8.3 Design Patterns Implemented 12
8.4 Database Used 13
9. Prototype 13
10. Testing & Evaluation 16
10.1 Black Box Testing 16
10.2 White Box Testing 18
10.3 Evaluation 18
11. Critical Thinking 19
11.1 Advantages 19
11.2 Limitations 19
11.3 Future Improvements 19

12. Conclusion 19
13. References 21
14. Marking Matrix with Self-Assessment 22
15. Appendices 23
Appendix A: Test screenshots 23
Appendix B: Source code repository 37

CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili

1. Introduction

A building that has than one lift requires a controller that decides which lift answers each call. This report talks about a desktop simulation of two lifts in an office building. It was written in C# with Windows Forms and a Microsoft Access database, for the unit CIS116-2. The report covers the planning, design, implementation, testing and evaluation of the program.

2. Problem Statement

The assignment brief requests a program that controls lifts in a building. The program must answer calls made from outside and inside each lift display the location of each lift animate the motion of each lift and record all lift events in a database. Because there are two lifts every outside call must be sent to the lift that can reach the requested floor the fastest while leaving any lift that is already traveling on its route untouched.

2.1 Major Challenges

- Determining which lift should answer a call.
- Avoiding changes to a trip that's already, in progress.
- Maintaining an user interface while the database is accessed.
- Ensuring the program runs on any computer by using paths and by not crashing if the database is missing.
- Organizing the code so that each class has a responsibility.

3. Proposed Solution

The assignment brief requests a program that controls lifts in a building. The program must answer calls made from outside and inside each lift display the location of each lift animate the motion of each lift and record all lift events in a database. Because there are two lifts every outside call must be sent to the lift that can reach the requested floor the fastest while leaving any lift that is already traveling on its route untouched.

2.1 Major Challenges

The assignment brief requests a program that controls lifts in a building. The program must answer calls made from outside and inside each lift display the location of each lift animate the motion of each lift and record all lift events in a database. Since there are two lifts, all external calls have to be directed to the lift that can take the passenger to their destination floor as quickly as possible without disturbing the lift on its way.

- Determining which lift should answer a call.
- Avoiding changes to a trip that's already, in progress.
- Maintaining an user interface while the database is accessed.
- Ensuring the program runs on any computer by using paths and by not crashing if the database is missing.
- Organizing the code so that each class has a responsibility.

4. Aims and Objectives

4.1 Aim

The main goal is to use C# Windows Forms and an Access database to design, develop and test a desktop application. The desktop application will manage two elevators in a two-floor building.

4.2 Objectives

- Develop a user interface with the call buttons, control panel for each elevator, status display of each floor, and a log.
- Develop animations for the elevators and their doors through timer and delegates.
- Process the inside calls through proper control mechanism.
- Store the events with timestamps in Access database through the model, relative path, and try-catch.
- Make use of BackgroundWorker for keeping GUI responsive.
- Implement the State and Observer patterns along with call allocation through shortest response time.
- Testing of the program.

5. Planning

5.1 Weekly Schedule

Week	Unit topic	Used in the project
1	Introduction to C#, .NET and the IDE	Set up Visual Studio and a first Windows Forms project
2	Object-oriented concepts in C#	Planned the classes: Lift, LiftDispatcher, states, observers
3	Delegates, timers and desktop applications	Timer-based animation and delegates (Task 4)
4	Data structures and ADO.NET	Queue and List for stops and jobs; DataTable for the log
5	Database connectivity	Access database and OleDbDataAdapter (Task 3)
6	Algorithms, design patterns, asynchronous programming	EstimateTime, State and Observer patterns, BackgroundWorker (Task 5)
7	Submission and demonstration	Testing, report, GitHub push, BREO upload and demonstration (Task 6)

Table 1: Weekly Schedule

5.2 Functional Requirements

- FR1: Call button is provided for each of the two floors.
- FR2: Control panel of each lift consists of two floor buttons and floor display.
- FR3: Floor status displays provide information about location of each lift.
- FR4: Call generated from outside is directed to the lift having shortest response time estimation.

- FR5: Request from inside the lift is served by this particular lift.
- FR6: Lifts and doors animate in their motion. Once the trip is started, it cannot be altered.
- FR7: Doors automatically open once lift reaches the floor. Doors automatically close. Open and Close buttons are provided to open and close the doors during lift's pause at the floor.
- FR8: Pressing emergency alarm button stops lifts from answering calls, completes existing trips, opens doors and logs an alarm.
- FR9: All events of lifts are stored with timestamps in the log. Log is visible on the GUI and updates in real time.
- FR10: Database actions are performed in the background. Loading bar appears once database is active.

5.3 Non-Functional Requirements

- Portability: the database is located with a path.
- Maintainability: database code is not. Each class has one job.
- Robustness: errors are caught with try-catch and shown as a message of a crash.
- Responsiveness: database work never blocks the window and animation runs at a 40 ms timer interval.
- Usability: displays and a layout that fills the window.
- Extensibility: a new state or observer can be added without changing the Lift class.

6. System/Technical Requirements

6.1 Hardware Requirements

- PC or laptop with at least 4GB of RAM (recommend 8GB)
- About 10GB of free disk for visual studio and project.

6.2 Software Requirements

- Windows 11
- Visual Studio with the .NET desktop development workload.
- Microsoft Access, or the 64-bit Access Database Engine, which provides the Microsoft.ACE.OLEDB provider.
- The ADO.NET OleDb classes (System.Data.OleDb).
- Git and a GitHub account for version control (GitHub Classroom 50).

7. System Design

7.1 Use Case Diagram

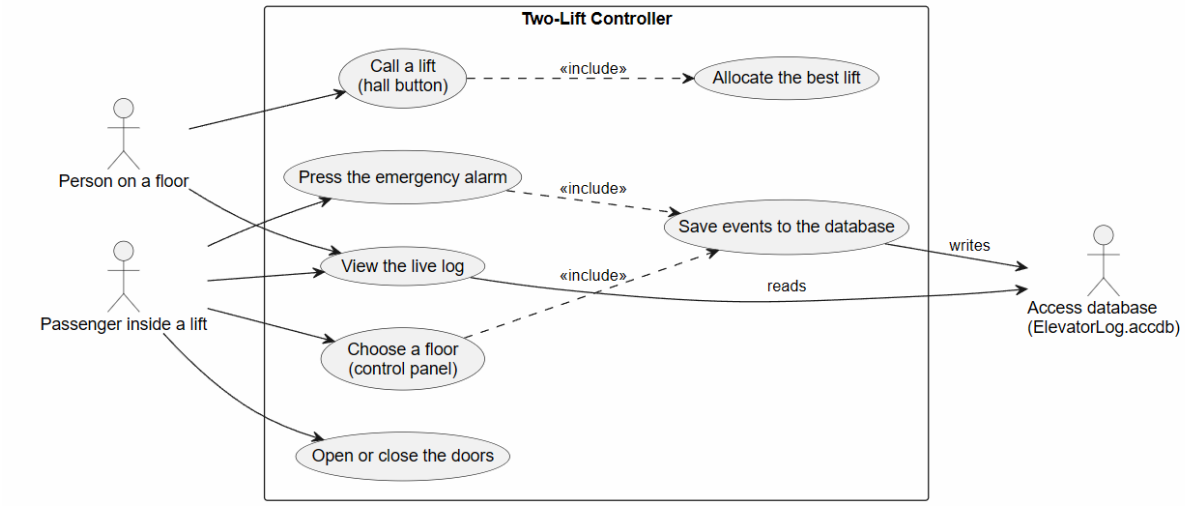


Diagram 1: Use case diagram

7.2 Class Diagram

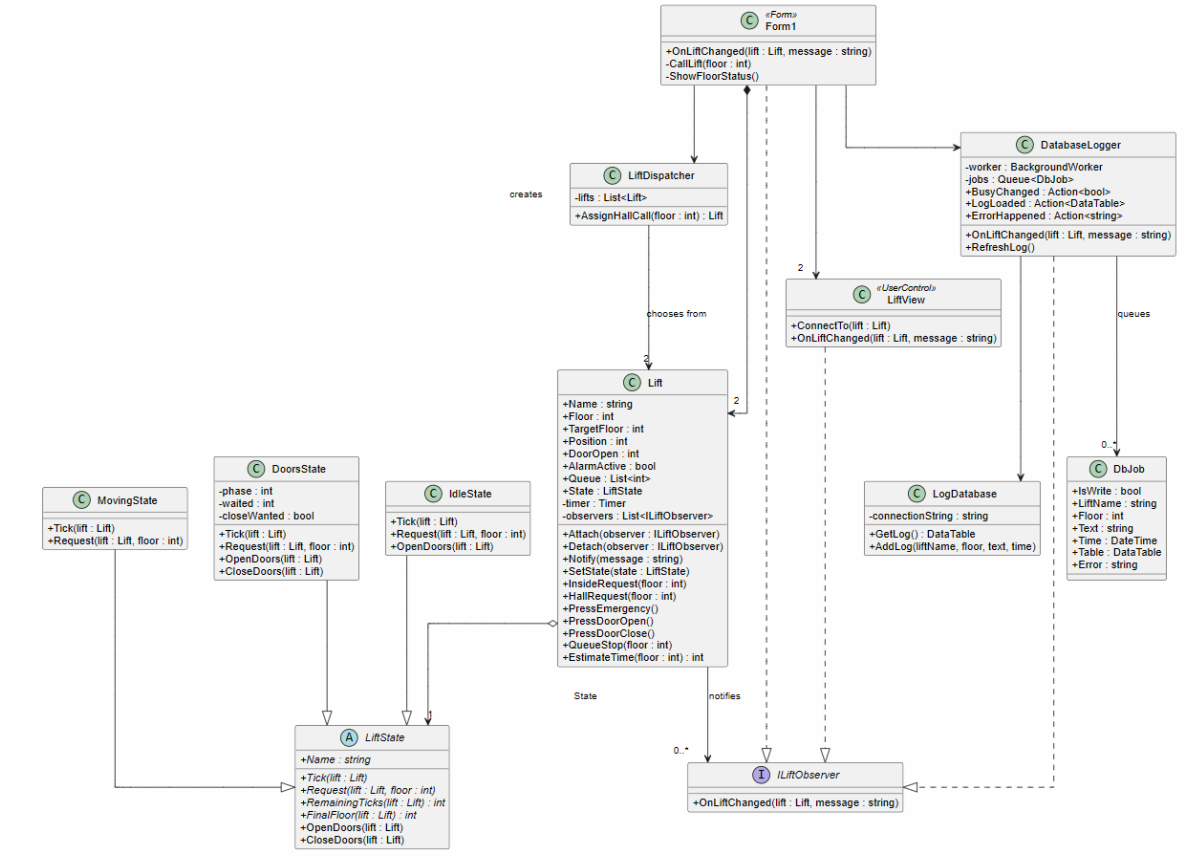


Diagram 2: Class diagram

7.3 ER Diagram

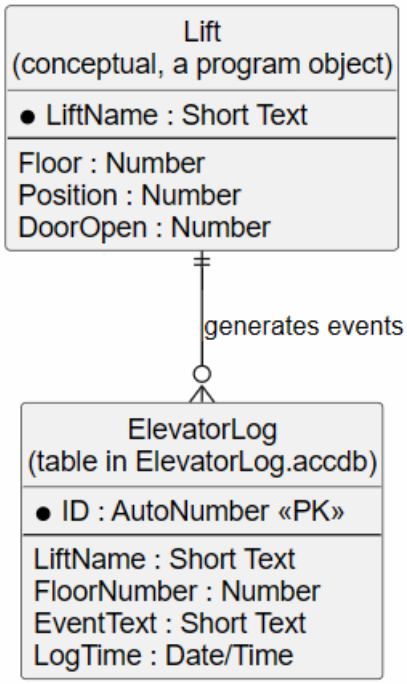


Diagram 3: ER diagram

7.4 Activity Diagram

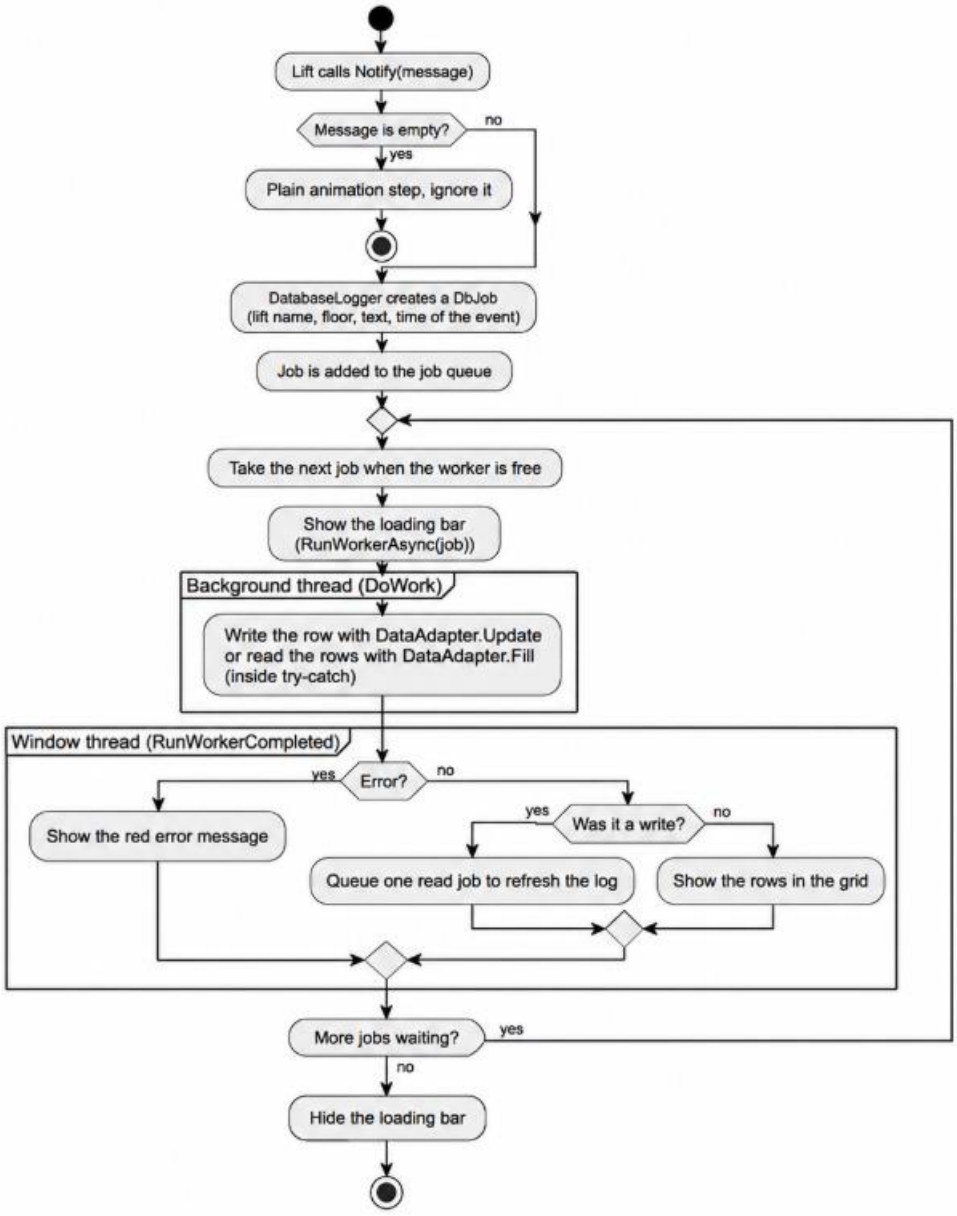


Diagram 4: Handling a call from outside the lifts

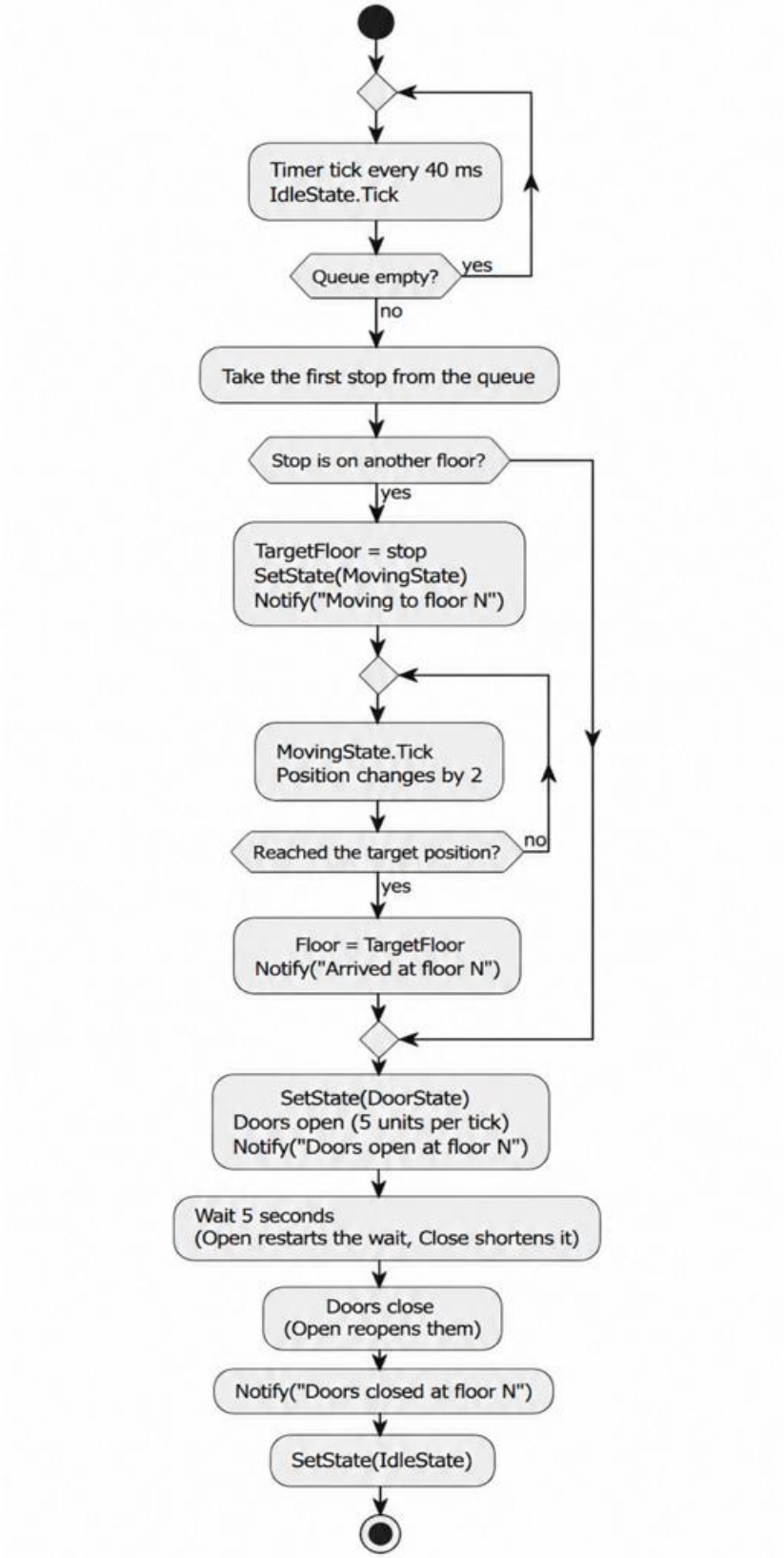


Diagram 5: Lift movement and doors, driven by the timer and the states

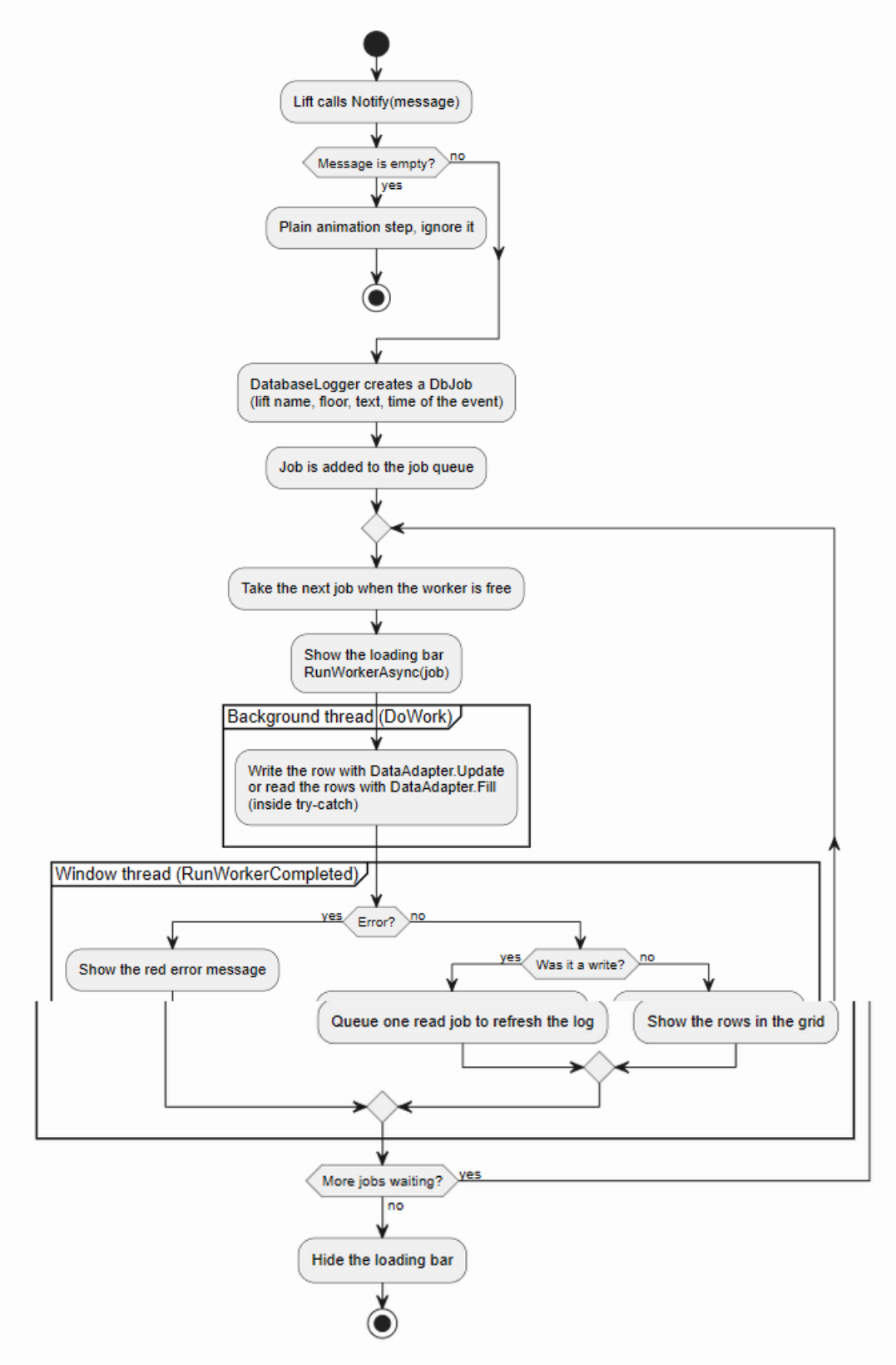


Diagram 6: Saving an event with the BackgroundWorker

8. Tech Stack Implementation

8.1 Language Used

The program is written in C# for.NET with Windows Forms as the assignment requires. C#'s strongly typed, supports object-oriented programming and Visual Studio gives a designer for forms. The animation uses System.Windows.Forms.Timer and the doors and the lift car are drawn with GDI+ (Graphics and LinearGradientBrush).

8.2 Object-Oriented Programming Implementation

Encapsulation: Data is private. Changed only through methods. In Lift, the Name, Queue and State properties have setters and the timer and the observer list are private fields. LogDatabase keeps its connection string private. Other classes change a Lift by calling methods such as InsideRequest or SetState.

Inheritance: IdleState, MovingState and DoorsState inherit from LiftState. LiftView inherits from UserControl Form1 from Form and BufferedPanel from Panel so they reuse what those classes already do.

Polymorphism: Lift calls State. Tick without knowing which state it holds. Each state overrides Tick so the same call does things. Notify also calls OnLiftChanged on objects of types, through the ILiftObserver interface.

Abstraction: LiftState is abstract. Iliftobserver is an interface. They say what must be done, not how. Lift only knows these abstractions so Lift does not depend on the screen or the database.

Event-driven programming and delegates: The program reacts to events: Button.Click, Timer.Tick, BackgroundWorker.DoWork and RunWorkerCompleted. The Tick event is based on a delegate. Databaselogger exposes three Action delegates (BusyChanged, LogLoaded and ErrorHappened) that the form fills in.

Resource management: Pens, brushes and fonts used when drawing are created in using blocks so they are released away. OleDbDataAdapter opens the database connection only while it fills or updates then closes it. Database jobs run one at a time so two threads never use the Access file together. BufferedPanel uses buffering to stop flicker.

8.3 Design Patterns Implemented

Observer pattern: Lift is the subject, with Attach, Detach and Notify. LiftView, Form1 and DatabaseLogger implement ILiftObserver. When a lift changes it calls Notify(message): an empty message is a plain animation step, and text is an event to log. The lift does not know who is listening, so a new observer can be added without changing Lift.

State pattern: LiftState is the abstract state. IdleState takes the next stop from the queue, MovingState moves the position, and DoorsState opens the doors, waits and closes them. A state moves the lift on with SetState. Requests are always added to the end of the queue, so a trip in progress is not disturbed. EstimateTime asks the current state for RemainingTicks and FinalFloor to work out the response time. The alarm is a flag on the lift, not a state.

Background processing: The logger is itself an observer, so saving events needs no code inside Lift. Combining Observer with a BackgroundWorker keeps the animation smooth. DoWork runs only database code on a second thread, and RunWorkerCompleted returns to the window thread, where it is safe to update the grid. Each job carries the time of the event, so the timestamp is correct even if the job waits in the queue.

Separation of responsibilities: Lift holds the data and the timing. The states hold the behaviour. LiftDispatcher chooses the lift. LiftView and Form1 only display. LogDatabase holds the database code. DatabaseLogger manages the background jobs. Each class has one job, which makes the code easier to explain, test and change.

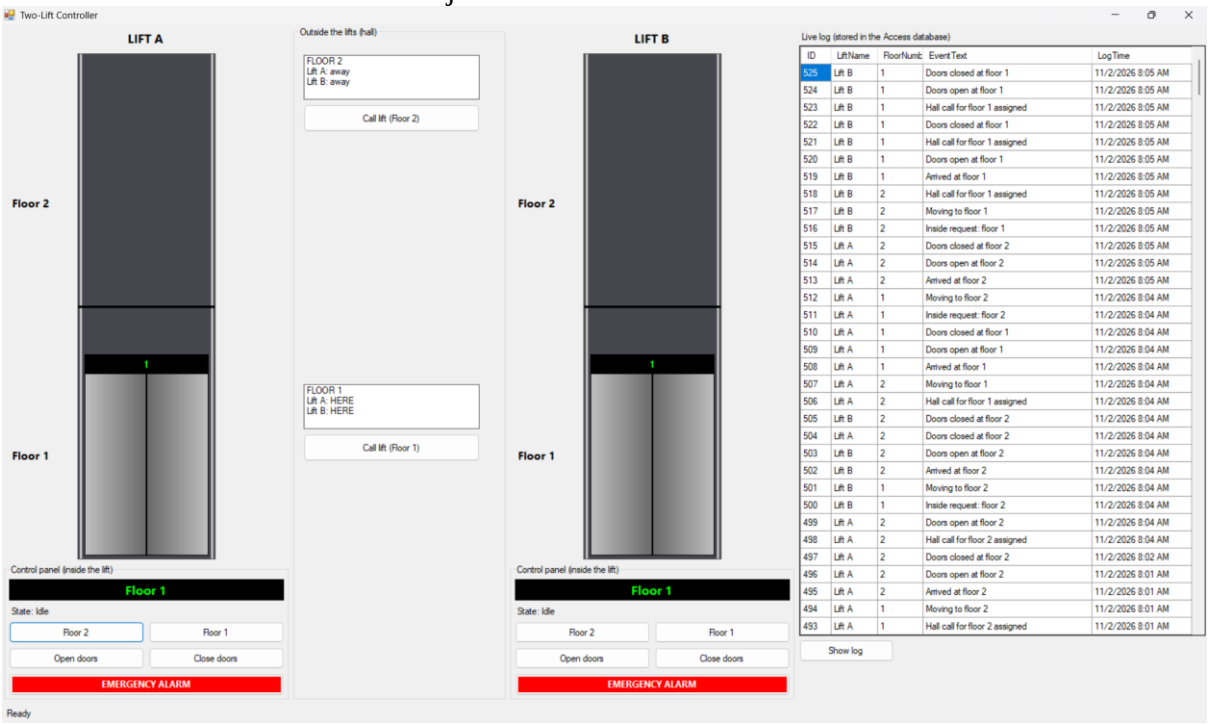
8.4 Database Used

Field	Type	Purpose
ID	AutoNumber (primary key)	Unique number of each log row
LiftName	Short Text	Which lift the event belongs to
FloorNumber	Number	The floor the lift was at when the event happened
EventText	Short Text	What happened, for example Arrived at floor 2
Longtime	Date/Time	When the event happened

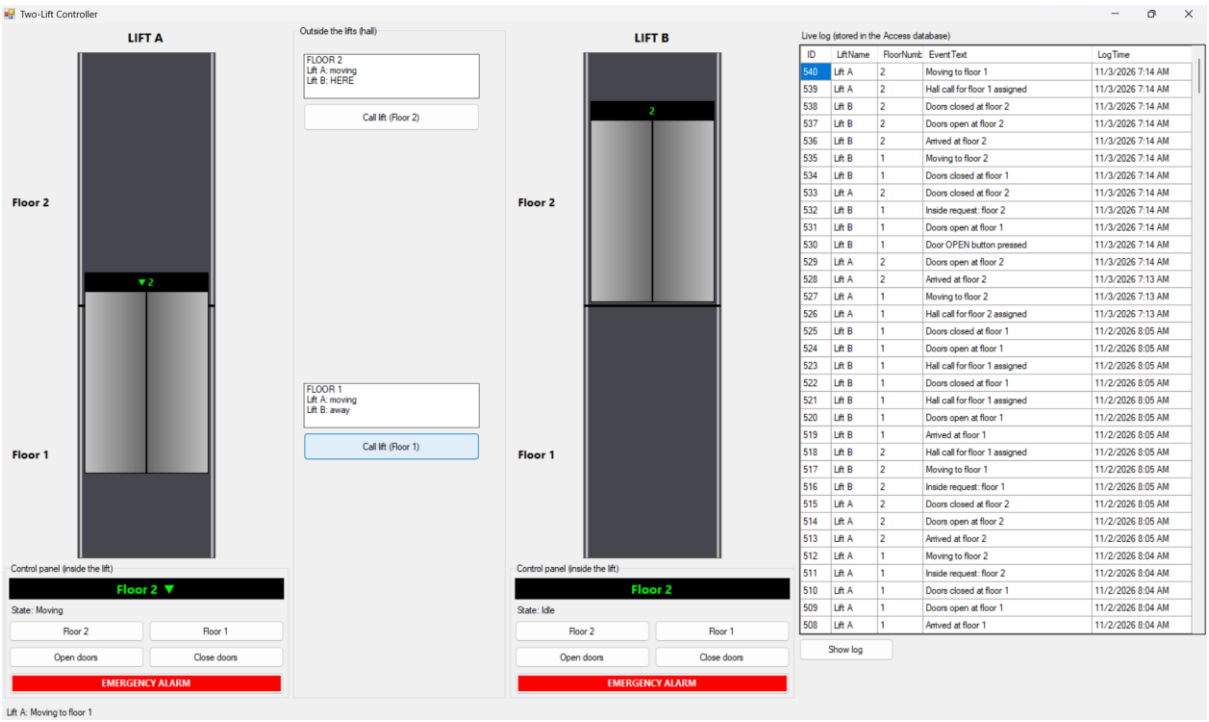
9. Prototype

The prototype is the program. The window fills the screen. Has four columns: Lift A, the hall panel, Lift B and the log. Each lift shows a shaft, a car with steel doors that open from the centre and a floor indicator. Under each lift is its control panel with a floor display a state line, two floor buttons, an Open doors button, a Close doors button and the red emergency alarm button. The hall panel has a call button and a status display for each floor. The log on the updates, by itself and a loading bar appears while the database is busy.

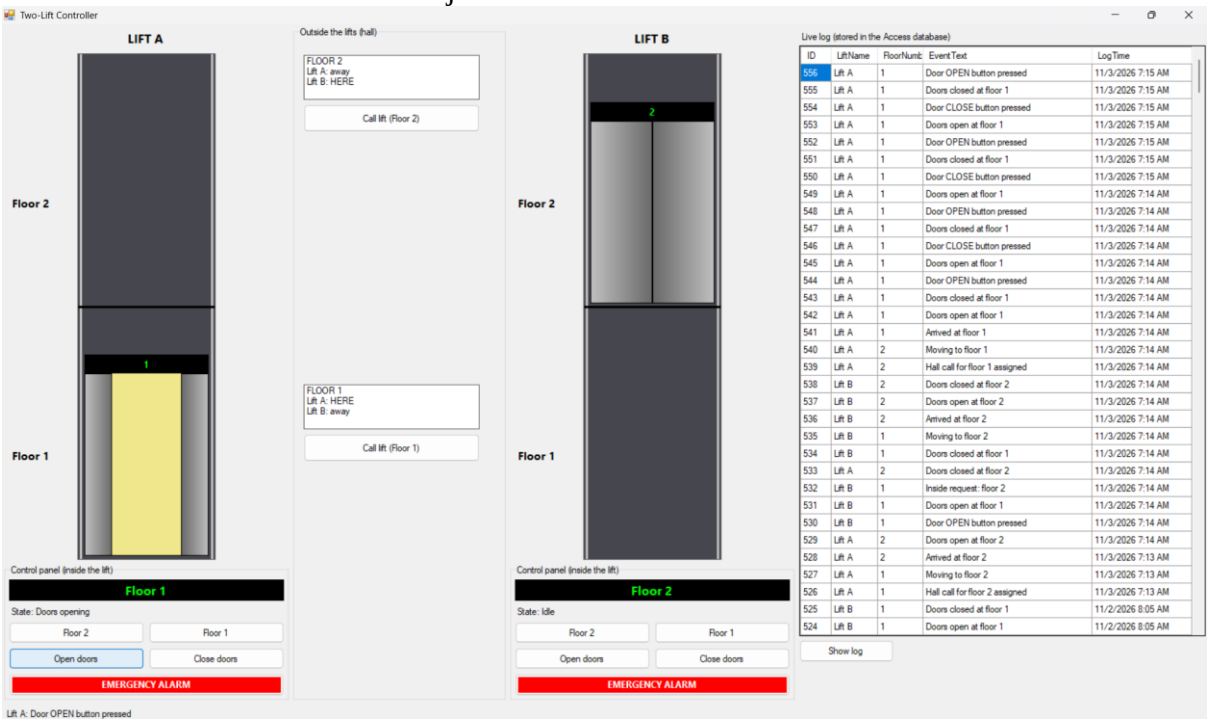
Prototype figure P1 - Main window at start-up



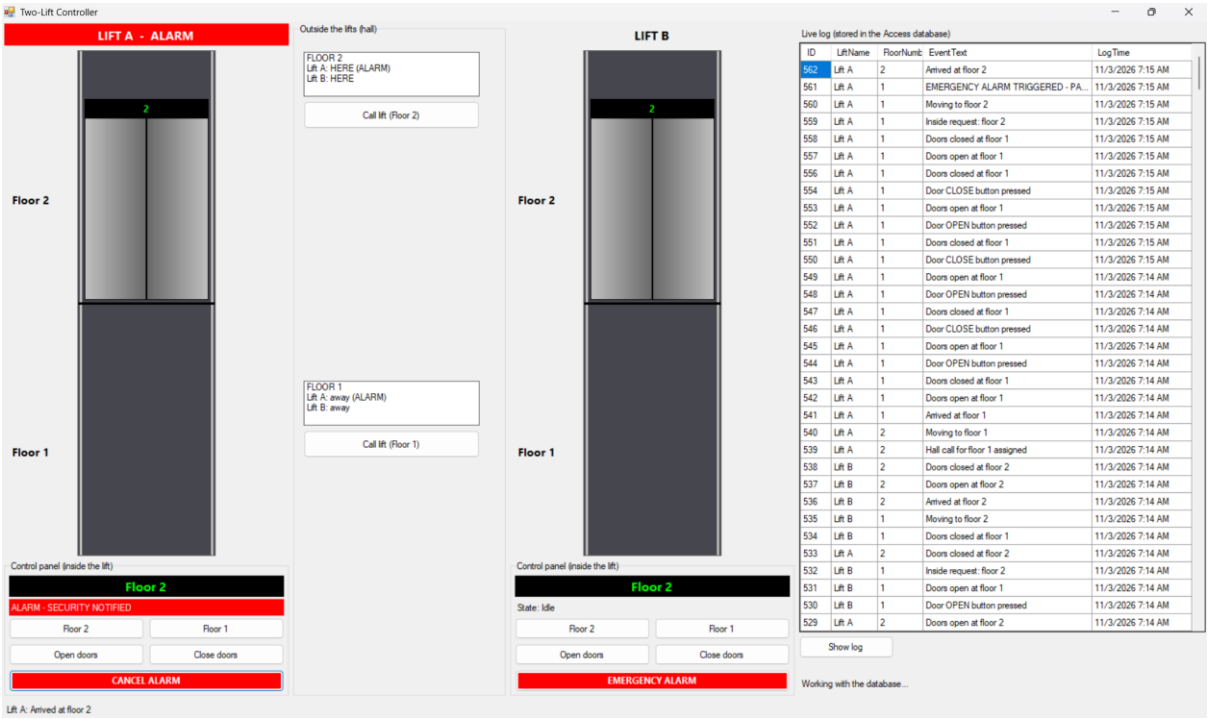
Prototype figure P2 - A lift moving between floor



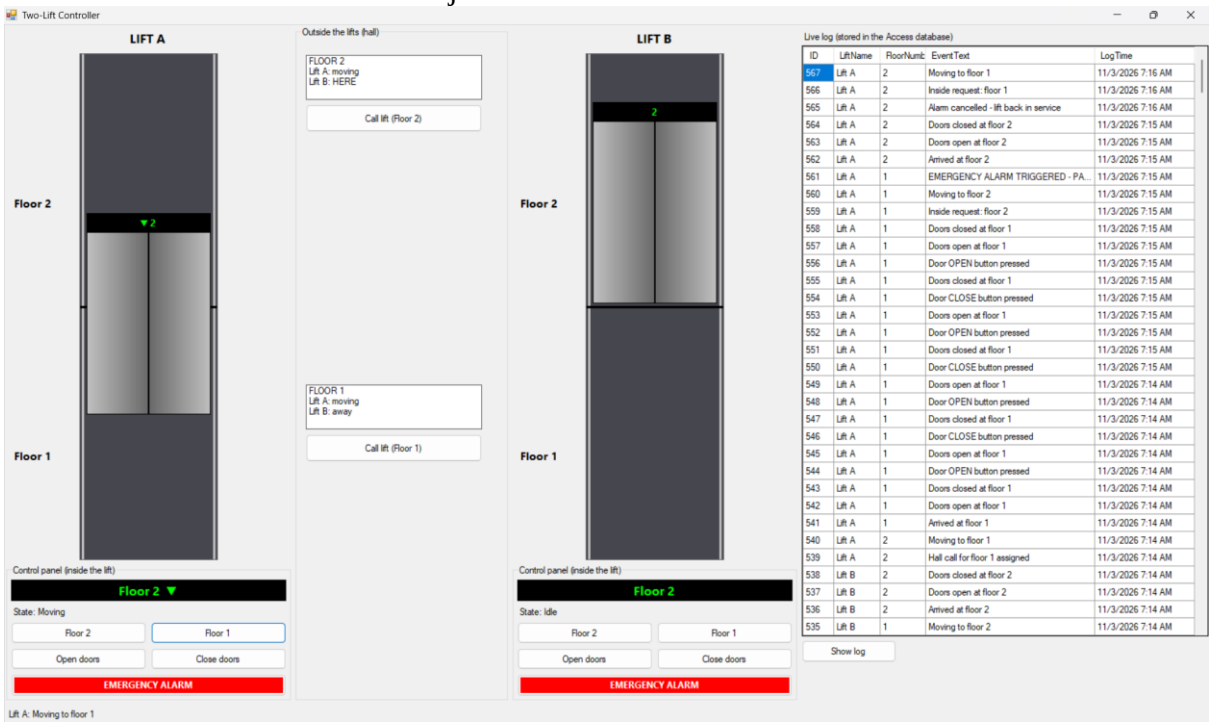
Prototype figure P3 - Doors open at a floor



Prototype figure P4 - Emergency alarm active



Prototype figure P5 - Live log with new rows



10. Testing & Evaluation

Tests were run by hand on Windows 11 with Visual Studio, in Debug|x64. The Fig. numbers in the tables refer to the screenshots in Appendix A.

10.1 Black Box Testing

Black box tests check what a user sees without looking at the code. All 15 tests were run and the results recorded.

#	What I did	Expected result	Actual result	Pass / Fail	Evidence
1	Start the application	Both lifts at Floor 1 with doors closed; old log rows load	Both lifts started at Floor 1 with doors closed. The live log loaded the earlier rows from the database.	Pass	Fig. 1
2	Press Call lift (Floor 2)	One lift moves up, doors open, floor displays update	One lift moved up to floor 2, its doors opened and the floor displays updated. New rows appeared in the log.	Pass	Fig. 2
3	Press Call lift (Floor 1) while the first lift is still moving	The other lift answers the call	The lift that was not moving answered the call. The log shows the hall call assigned to the other lift.	Pass	Fig. 3
4	Press a floor button inside one lift	Only that lift moves	Only the lift whose panel I used moved. The other lift stayed where it was.	Pass	Fig. 4
5	Press the floor button for the floor the lift is already on	Doors open, no crash	The doors opened and the program carried on normally.	Pass	Fig. 5
6	Press EMERGENCY ALARM while a lift is moving	Lift keeps moving, arrives, opens doors; red alert flashes; alarm line is logged	The lift kept moving, arrived and opened its doors. The title and status text flashed red and the log has the line EMERGENCY ALARM TRIGGERED - PASSENGER REQUESTING ASSISTANCE.	Pass	Fig. 6
7	With one alarm active, call a lift from the hall	The other lift answers	The lift with no alarm answered the call.	Pass	Fig. 7
8	Press CANCEL ALARM	Lift returns to normal service	The alarm cleared and the lift worked normally again. The log shows Alarm cancelled - lift back in service.	Pass	Fig. 8
9	Press Open doors while the doors are closing	Doors reopen	At first the doors only opened again after they had fully closed (see defect 1). After the fix they reversed straight away.	Pass	Fig. 9
10	Press Close doors while the doors are waiting open	Doors close early	At first the Close press was ignored while the doors were still opening (see defect 1). After the fix the doors closed early.	Pass	Fig. 10
11	Press Open doors while the lift is moving	Ignored; the log says so	Nothing happened to the doors. The log says Door open ignored: lift is moving.	Pass	Fig. 11
12	Check the log after a trip	Rows appear automatically with timestamps	New rows appeared in the grid without pressing Show log, each with the lift name, floor, event text and time.	Pass	Fig. 12
13	Close and reopen the application	Earlier log rows are still there	The earlier rows were still in the log after restarting.	Pass	Fig. 13
14	Temporarily rename ElevatorLog.accdb in the output folder, then use the lifts	A red database message appears and the program does not crash	A red database error message appeared and the lifts kept working.	Pass	Fig. 14
15	Extract the zip into a different folder and run it	It works, which shows the relative path is correct	The program ran from the new folder and the log loaded, so the relative path works.	Pass	Fig. 15

Defects found and how they were fixed

#	What I saw	Why it happened	How I fixed it
1	In tests 9 and 10 the Open and Close door buttons seemed to act only after the doors had fully closed or fully opened. The log showed my Close press arriving while the doors were still opening.	The doors waited only 1 second and closed in under a second, so the lift was often already idle when I pressed Open. The Close button was only handled while the doors were waiting open, so it was ignored while they were opening.	I made the doors wait 5 seconds and changed DoorTicks in Lift to 165 to match, so the response time estimate stays correct. DoorsState now remembers a Close press during opening and closes once the doors are fully open.
2	The log stayed empty and a red message said the Microsoft.ACE.OLEDB.12.0 provider is not registered on the local machine.	The program was running as 32-bit but the Access database engine on my laptop is 64-bit, so Windows could not find the provider.	I changed the build setting to Debug|x64 and rebuilt the solution. The log then loaded normally. This also showed that the exception handling works, because the lifts kept running and an error message was shown instead of a crash.
3	On screen the alarm button was cut off at the bottom of each control panel, the doors were not in the middle of the car, and the car was wide and flat, like a cupboard.	The layout used fixed sizes and bottom anchors that did not suit my screen scaling (125%), and the car size was taken from the whole shaft.	I changed the layout to docked panels and a table layout, calculated the car and door sizes from the real control sizes, and drew the doors so they open from the centre.

with the State and Observer patterns. The self-assessment in section 14 is based on these results. The main weakness of the testing is that it was manual, so a future version should add unit tests.

10.2 White Box Testing

Code path	Covered by	What was seen	Result
LiftDispatcher.AssignHallCall picks the lift with the lowest EstimateTime	Tests 2 and 3	A hall call went to the lift that could answer soonest	Pass
EstimateTime returns NotAvailable when AlarmActive is true	Test 7	The lift with an alarm was skipped and the other lift answered	Pass
IdleState.Tick: next stop equals the current floor, so only the doors open	Test 5	Doors opened without the lift moving	Pass
Lift.PressDoorOpen ignores the button while the state is MovingState	Test 11	The doors did not open and the log says the press was ignored	Pass
DoorsState phases: opening, waiting, closing, and the Open and Close handling in each phase	Tests 9 and 10	The wrong behaviour was found and fixed (defect 1)	Pass (after fix)
Lift.PressEmergency sets AlarmActive, clears the queue and logs the alarm; pressing again clears it	Tests 6 and 8	The trip finished, the alarm text flashed, and the alarm was cleared	Pass
Notify calls every attached observer (LiftView, Form1, DatabaseLogger)	Tests 2 and 12	Screen, floor status displays and log all changed from one Notify call	Pass
DatabaseLogger: write job, then a read job to refresh; the error branch when job. Error is set	Tests 12 and 14	New rows appeared automatically; a missing database gave a red message	Pass
LogDatabase builds the path from AppContext.BaseDirectory	Test 15	The program worked from a different folder	Pass

10.3 Evaluation

All fifteen black box tests were successful, two required fixes. Three flaws were revealed during testing; these defects are explained above. The application meets all requirements of the specifications, including GUI, control logic, DataAdapter.Update and the database, animation, BackgroundWorker and the dual lifts that use State and Observer patterns. Self-assessment in section fourteen is based on the results.The major drawback of testing is that it was conducted manually and the future release should include unit tests.

11. Critical Thinking

11.1 Advantages

- The design is explicit. There is one role for each class.
- Scalable: another lift is another Lift object; new behaviour is a state class and a new observer does not require any changes in Lift.
- The window is responsive since the database queries are performed in the background.
- Relative path and error handling ensure that the program is portable and robust.
- Doors, the floor indicator and the alarm simulate a real lift better.
- Complete event history is stored in the database.

11.2 Limitations

- Two floors only are implemented.
- EstimateTime uses predefined times. It ignores the travel direction and the number of passengers.
- When the alarm is pressed, the lift waiting is stopped, along with all hall calls, which are not provided to the other lift.
- The log does not have any limits, each refresh reads all rows.
- Windows and 64-bit Access Database Engine are required.
- No automated unit tests were created.
- An alarm just gives a message and plays a sound. No notification to others.

11.3 Future Improvements

- Give the cleared hall calls to the lift when the alarm is pressed.
- More floors and direction-based scheduling should be supported.
- Automated unit tests for the dispatcher and the states should be added.
- Log filtering, searching and exporting should be implemented.
- Set the door opening time and lift speed.
- Use the database that requires no engine, for example SQLite.

12. Conclusion

This project provides the working controller of two lifts that fulfils all requirements of the assignment brief. The State pattern encapsulates the behaviour of each lift in classes, the Observer pattern allows one lift to update the interface, status displays and the database without knowing about them, and the BackgroundWorker makes the window responsive while events are being saved. Three defects were detected in testing, and they are described in section 10.

13. References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design patterns: elements of reusable object-oriented software. Reading, MA: Addison-Wesley.

Microsoft (n.d.) BackgroundWorker Class. Microsoft Learn. Available at: https://learn.microsoft.com.

Microsoft (n.d.) OleDbDataAdapter Class. Microsoft Learn. Available at: https://learn.microsoft.com.

Microsoft (n.d.) Timer Class (System.Windows.Forms). Microsoft Learn. Available at: https://learn.microsoft.com

Microsoft (n.d.) Timer Class (System.Windows.Forms). Microsoft Learn. Available at: https://learn.microsoft.com

14. Marking Matrix with Self-Assessment

Each item is marked Yes because the tests in section 10 passed. Mark awarded is left for the lecturer.

Task	Sub-task	Possible marks	Self-assessment (Yes/No)	Reference to testing report	Mark awarded
Task 1	Complete GUI for Task 1	10	Yes	Tests 1, 2, 4	
	Event handler skeleton implemented for all GUI buttons	10	Yes	Tests 2, 4, 5, 9, 10, 12	
Task 2	All event handlers function as specified	10	Yes	Tests 2, 3, 4, 5	
Task 3	Database designed and successfully connected	5	Yes	Tests 1, 12	
	Log information retrieved from DB and displayed in GUI	5	Yes	Tests 1, 12, 13	
	Log information stored in DB	5	Yes	Tests 12, 13	
	Disconnected model used (via DataAdapter.Update() instead of ExecuteNonQuery())	5	Yes	Test 12; section 8.4	
	Relative paths used; logical errors eliminated; exceptions handled with try-catch	5	Yes	Tests 14, 15; defect 2	
	No duplication in DB-related event handlers	5	Yes	Section 8.4 (one MakeAdapter method, one DatabaseLogger)	
Task 4	Events described in Task 2 animated using delegation and timer	10	Yes	Tests 2, 3, 4	
Task 5	Application efficiency improved by separating GUI and DB operations using BackgroundWorker	5	Yes	Tests 12, 14	
	System extended to support two elevators; State and Observer Patterns applied to manage elevator logic	15	Yes	Tests 3, 4, 6, 7; section 8.3	
Task 6	Testing report includes all expected content	10	Yes	Section 10	
Total		100			

15. Appendices

Appendix A: Test screenshots

Figure 1 - Test 1: Start the application

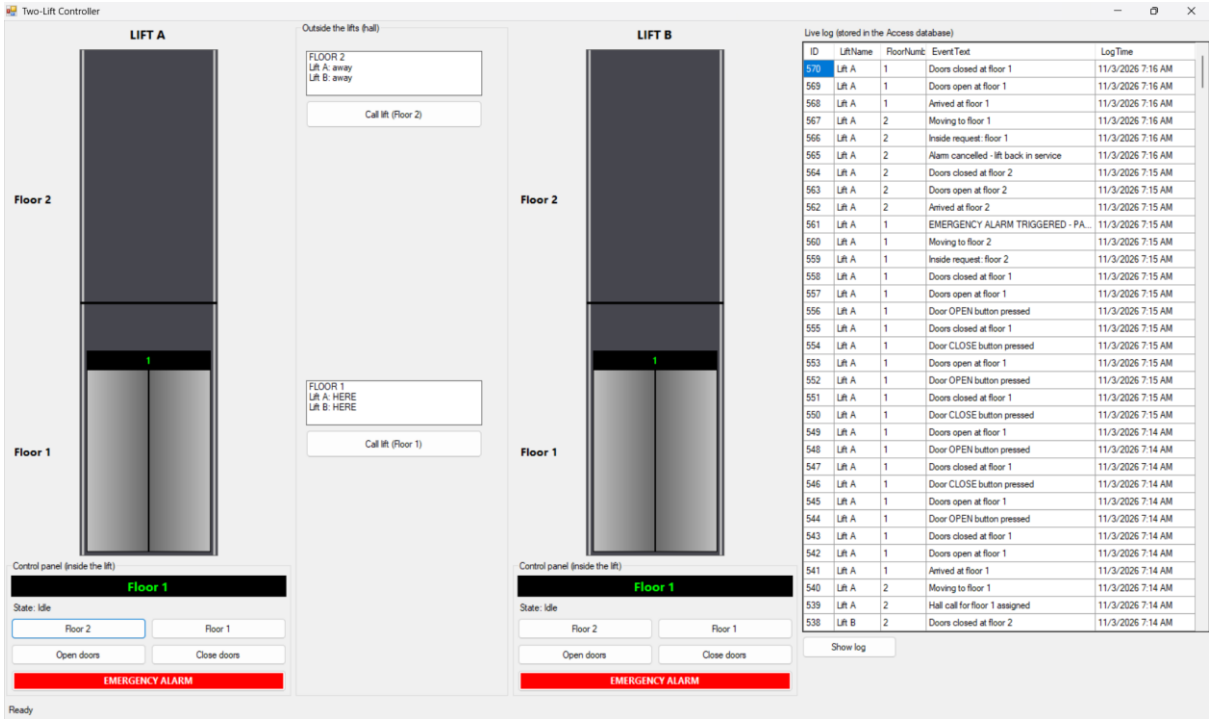


Figure 2 - Test 2: Press Call lift (Floor 2)

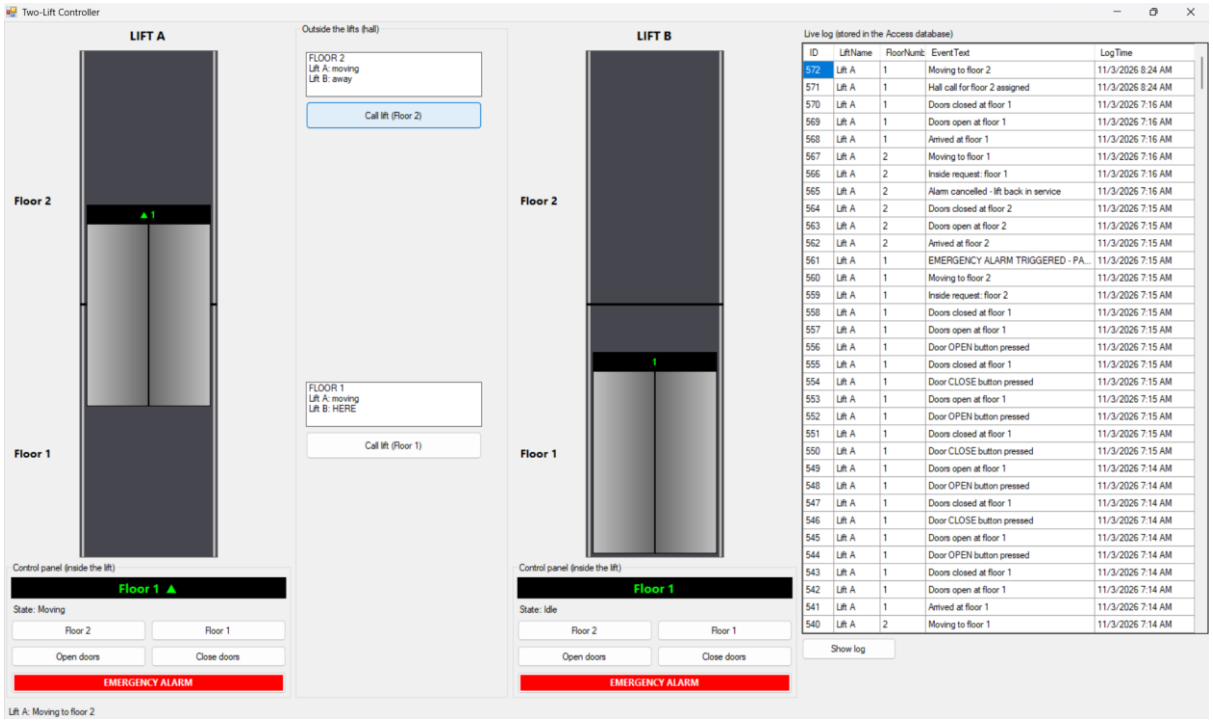


Figure 3 - Test 3: Press Call lift (Floor 1) while the first lift is still moving

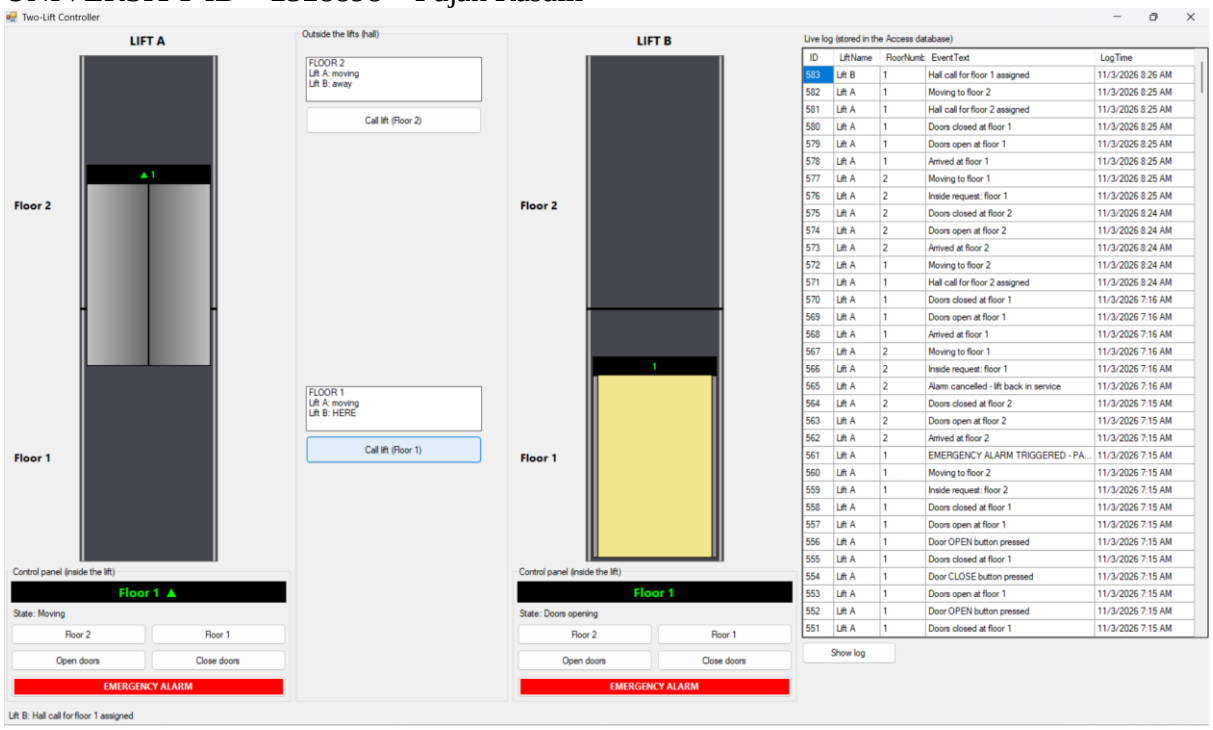


Figure 4 - Test 4: Press a floor button inside one lift

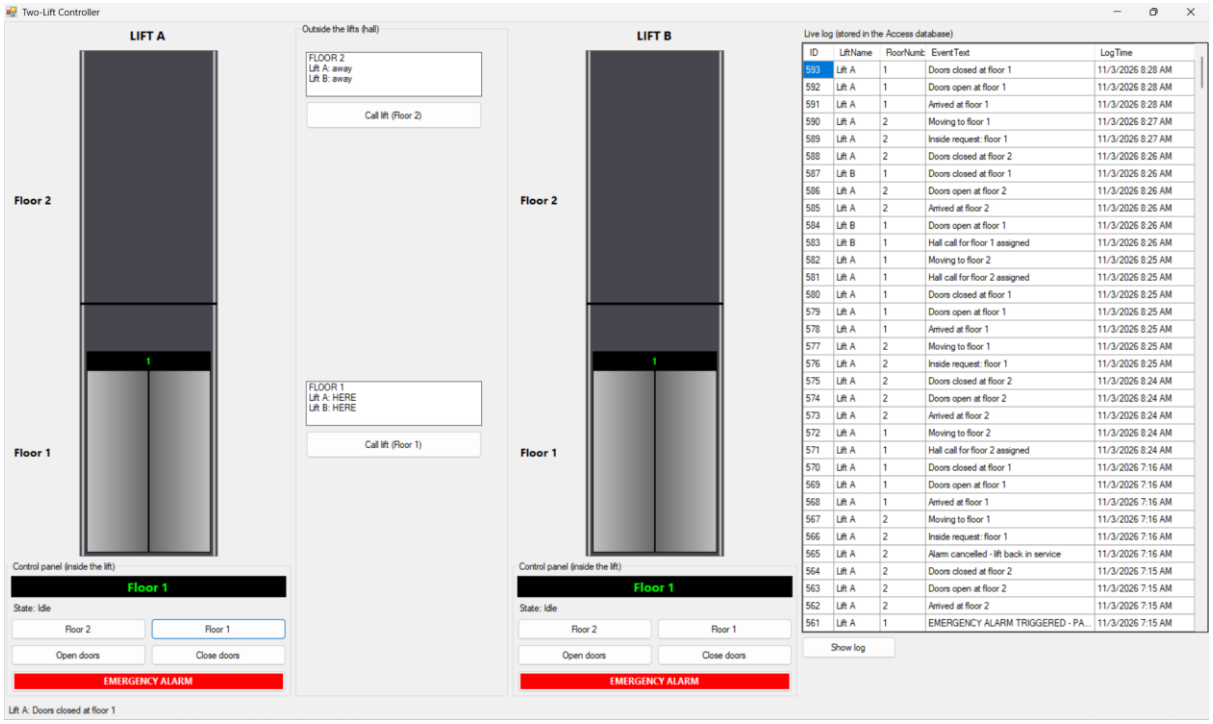


Figure 5 - Test 5: Press the floor button for the floor the lift is already on

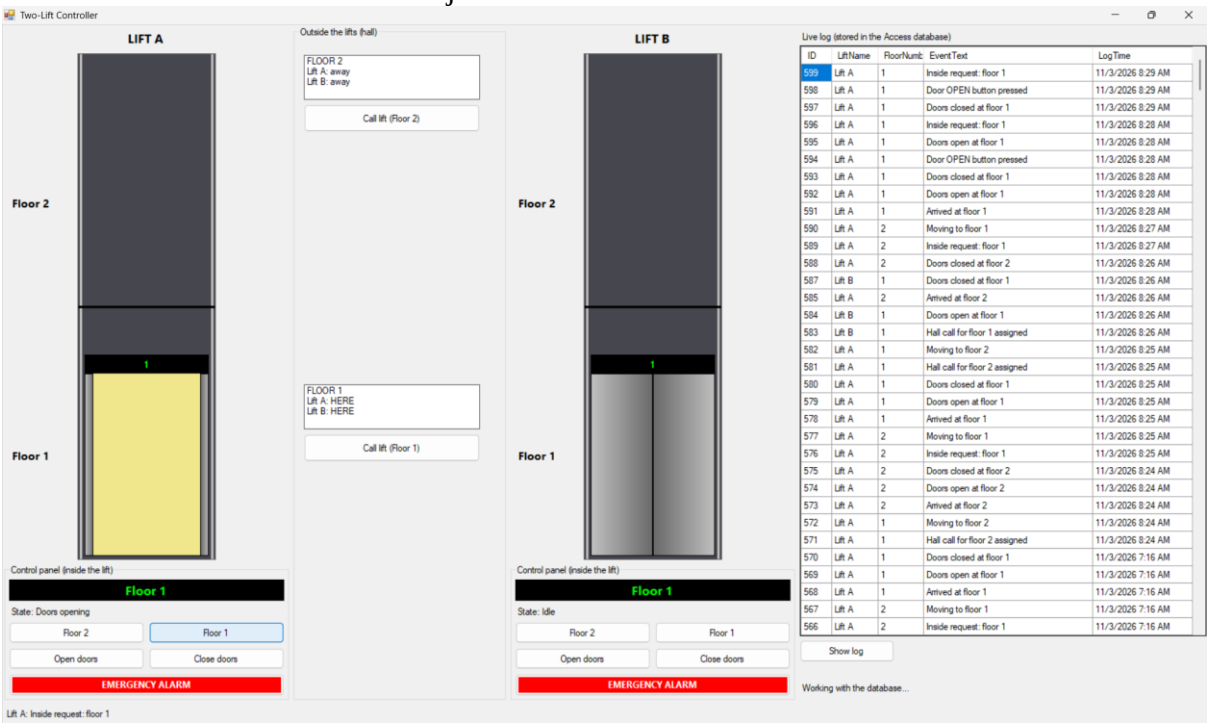


Figure 6 - Test 6: Press EMERGENCY ALARM while a lift is moving

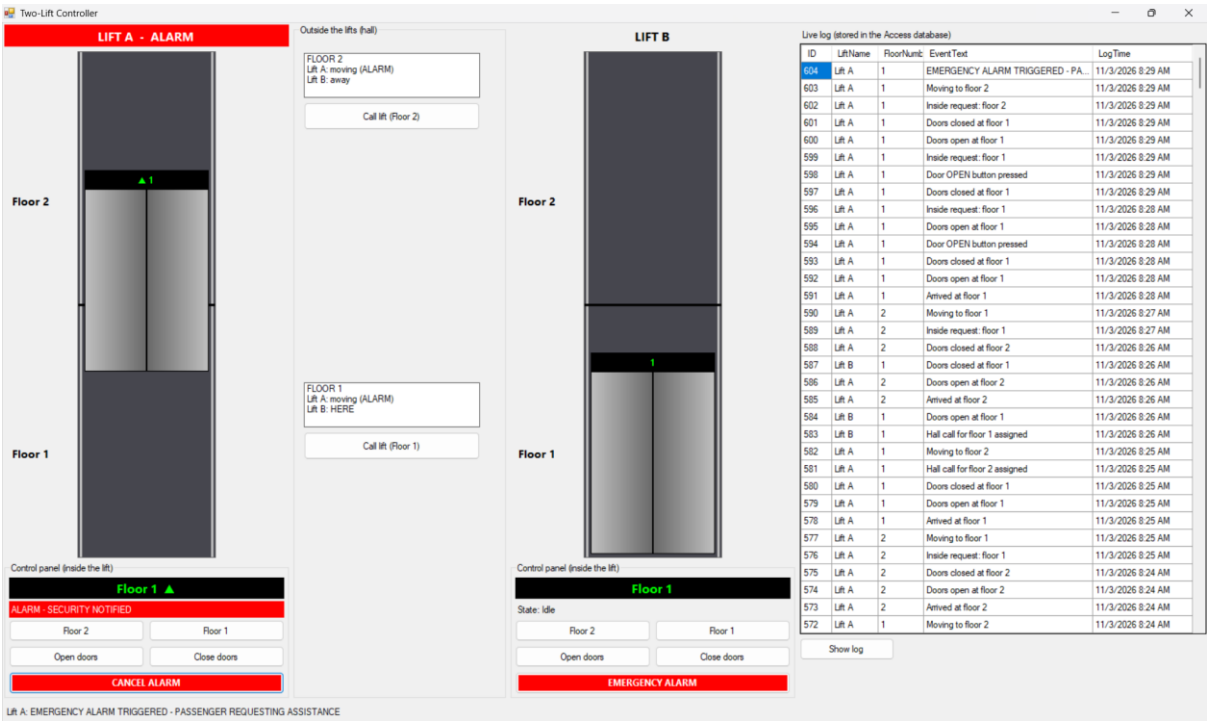


Figure 7 - Test 7: With one alarm active, call a lift from the hall

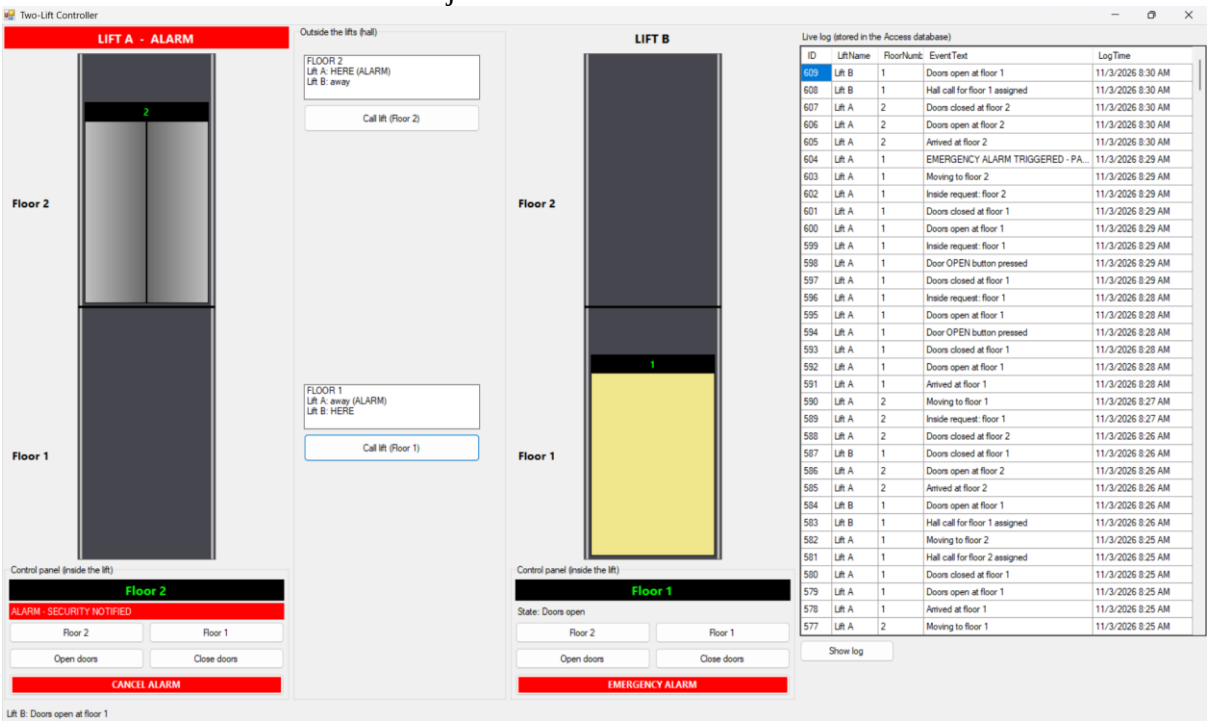


Figure 8 - Test 8: Press CANCEL ALARM

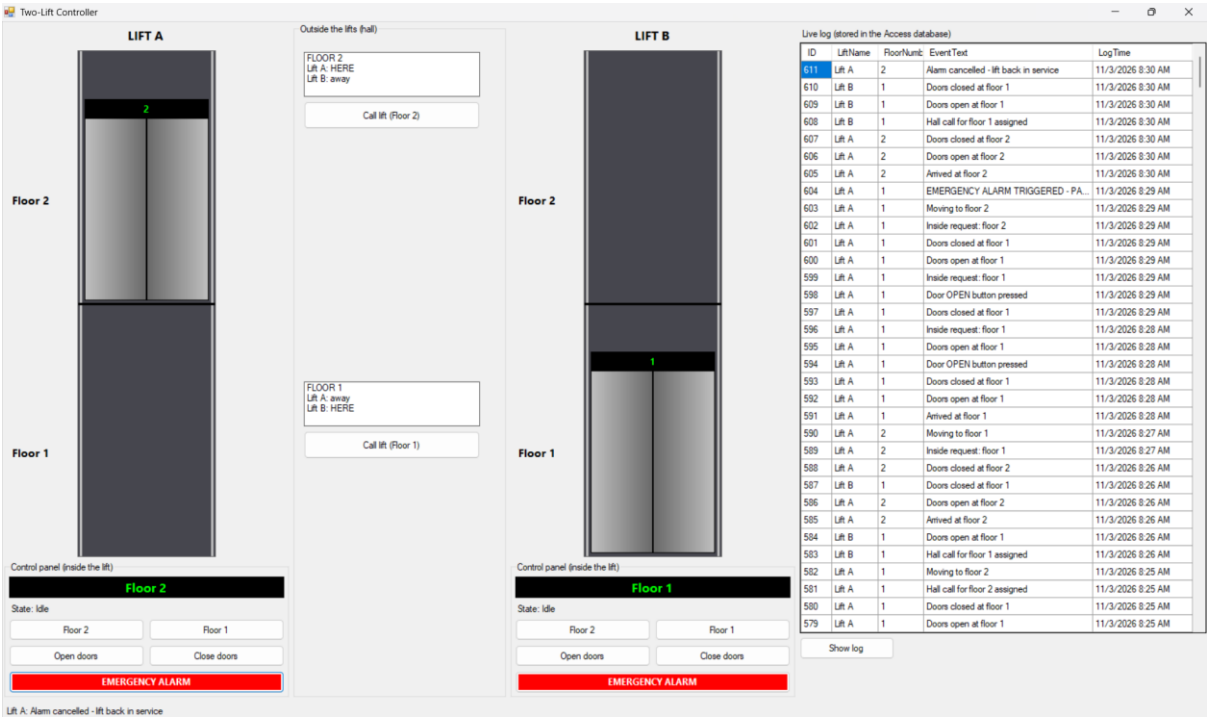


Figure 9 - Test 9: Press Open doors while the doors are closing

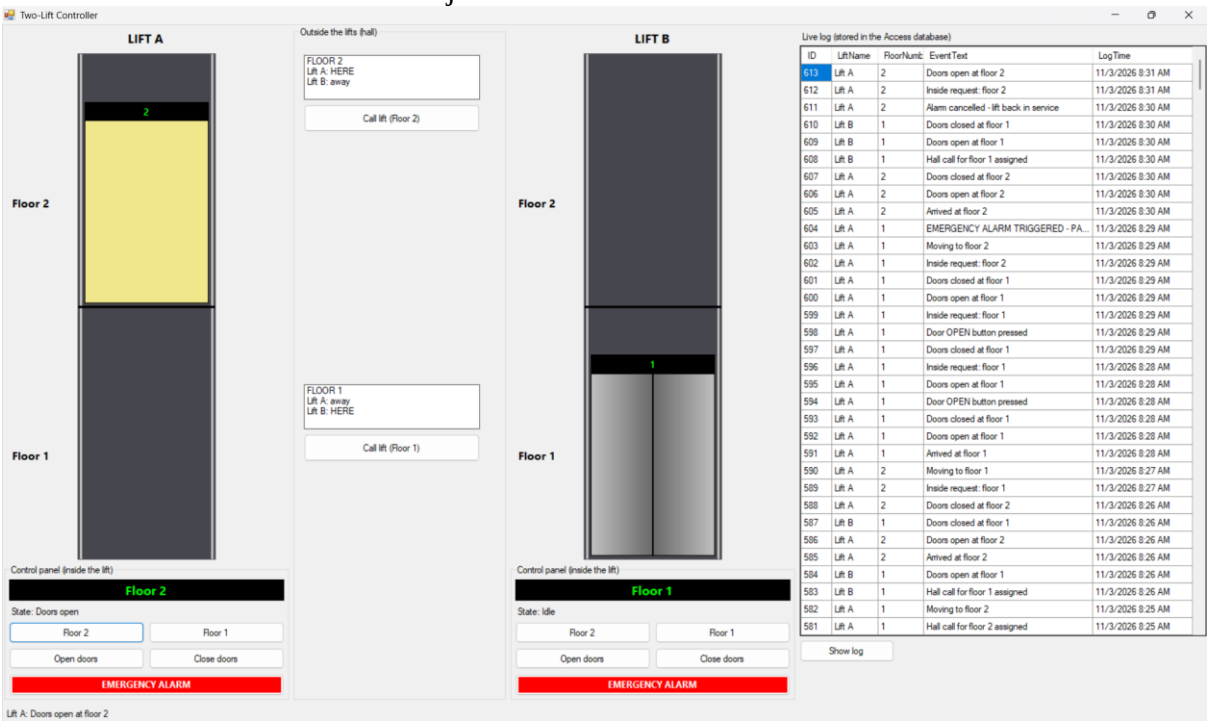


Figure 10 - Test 10: Press Close doors while the doors are waiting open

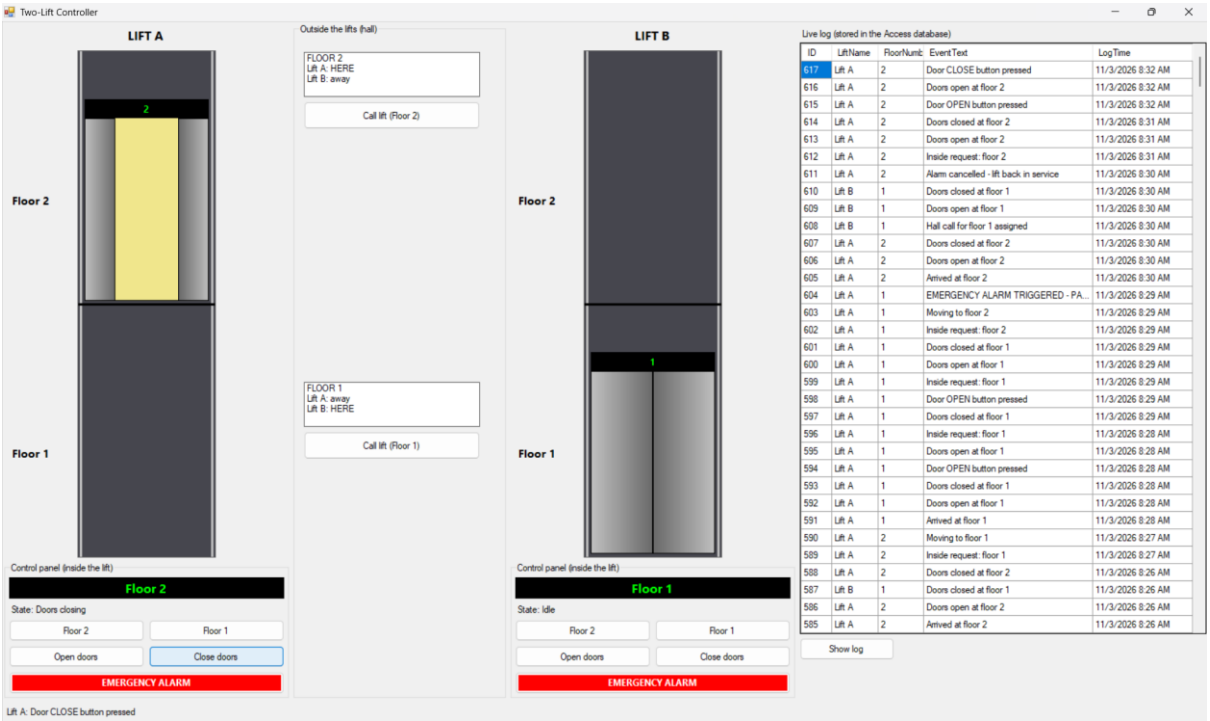


Figure 11 - Test 11: Press Open doors while the lift is moving

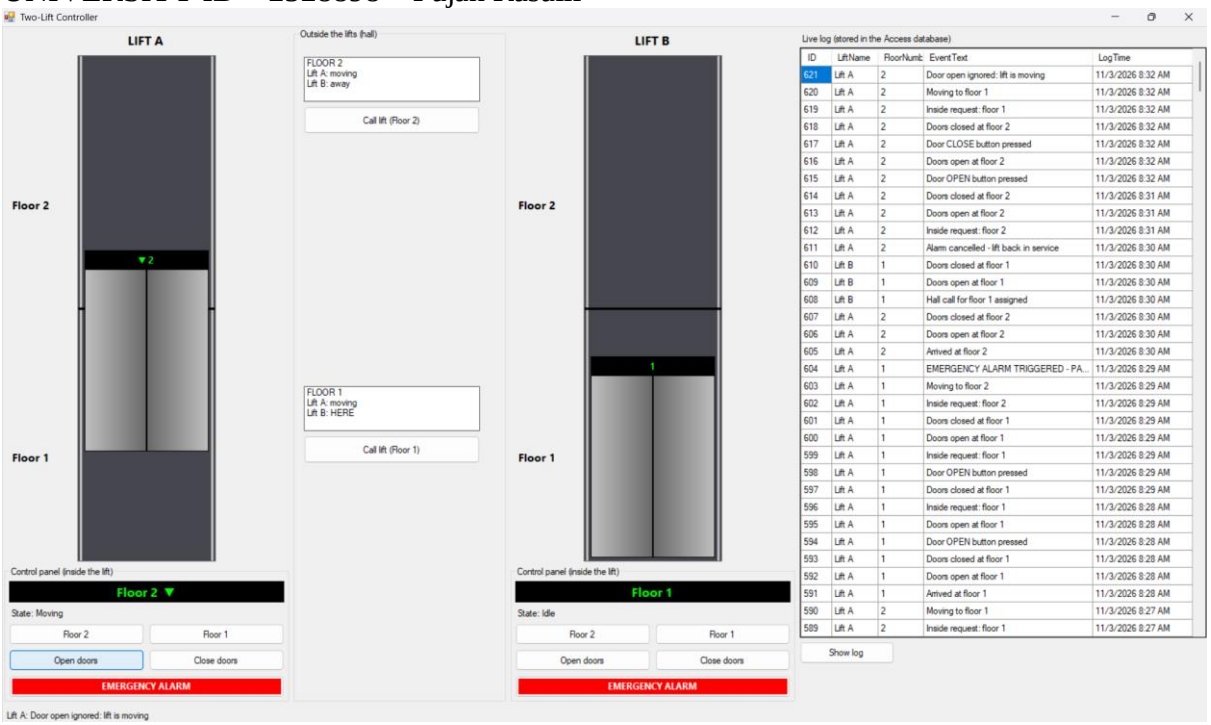


Figure 12 - Test 12: Check the log after a trip

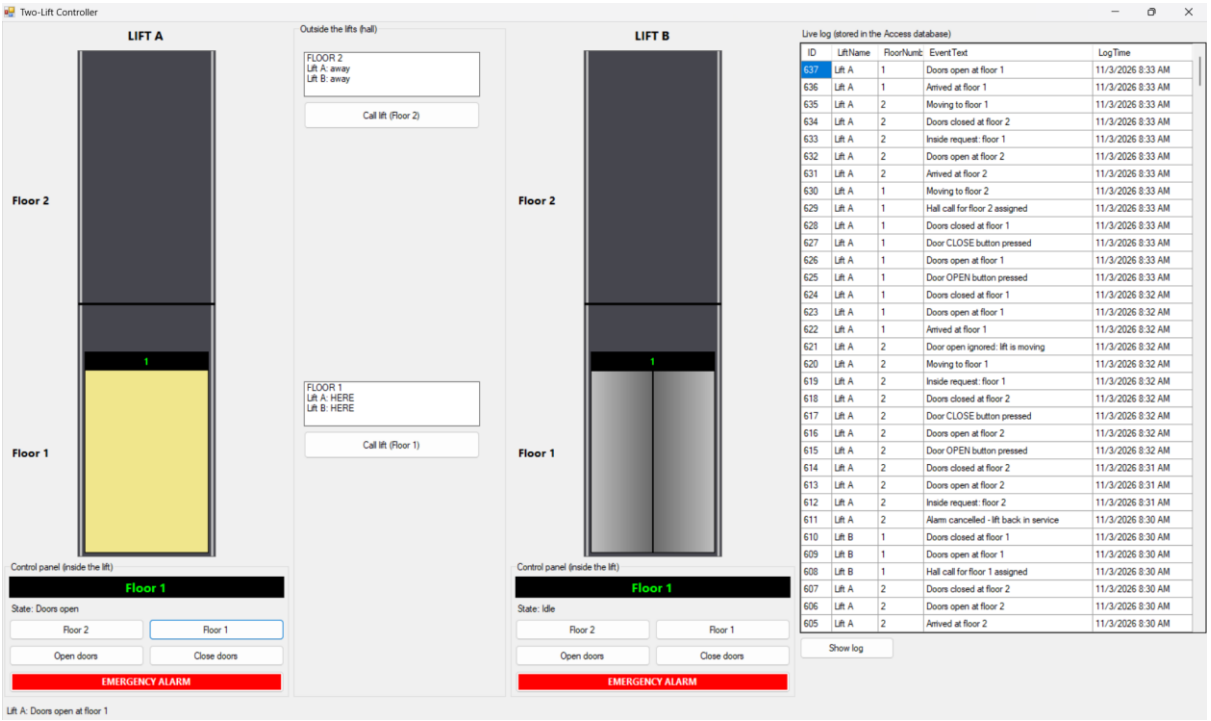


Figure 13 - Test 13: Close and reopen the application

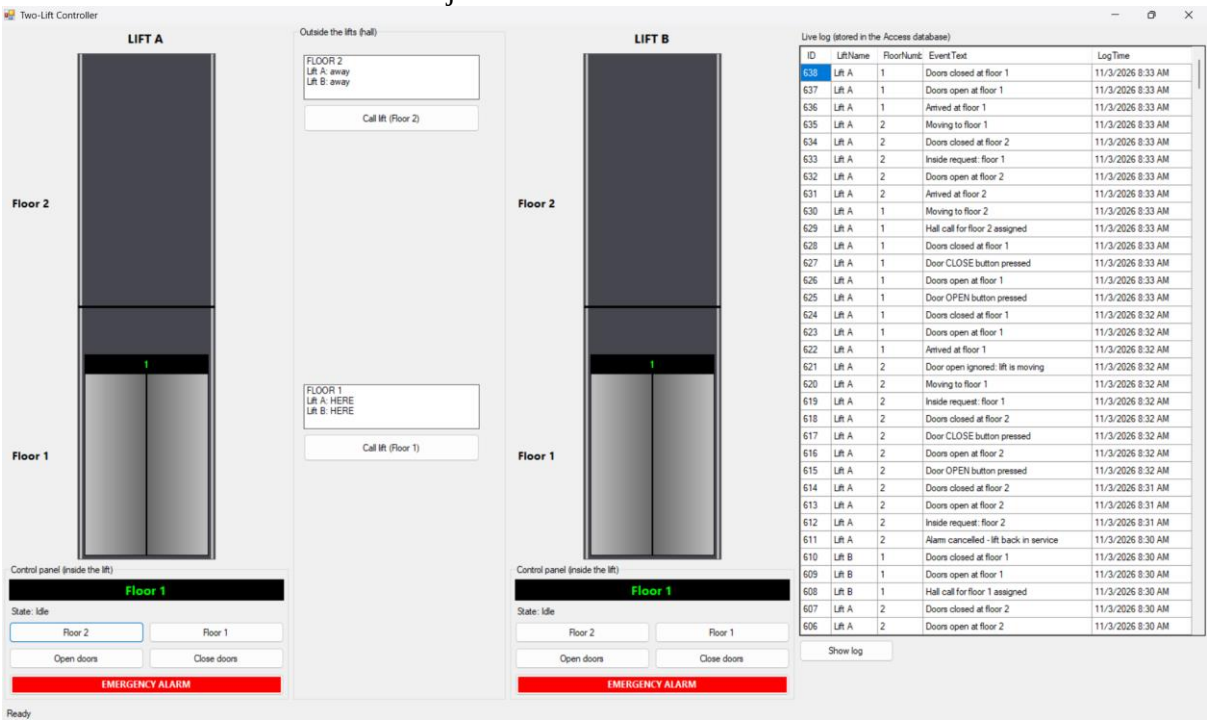


Figure 14 - Test 14: Temporarily rename ElevatorLog.accdb in the output folder, then use the lifts

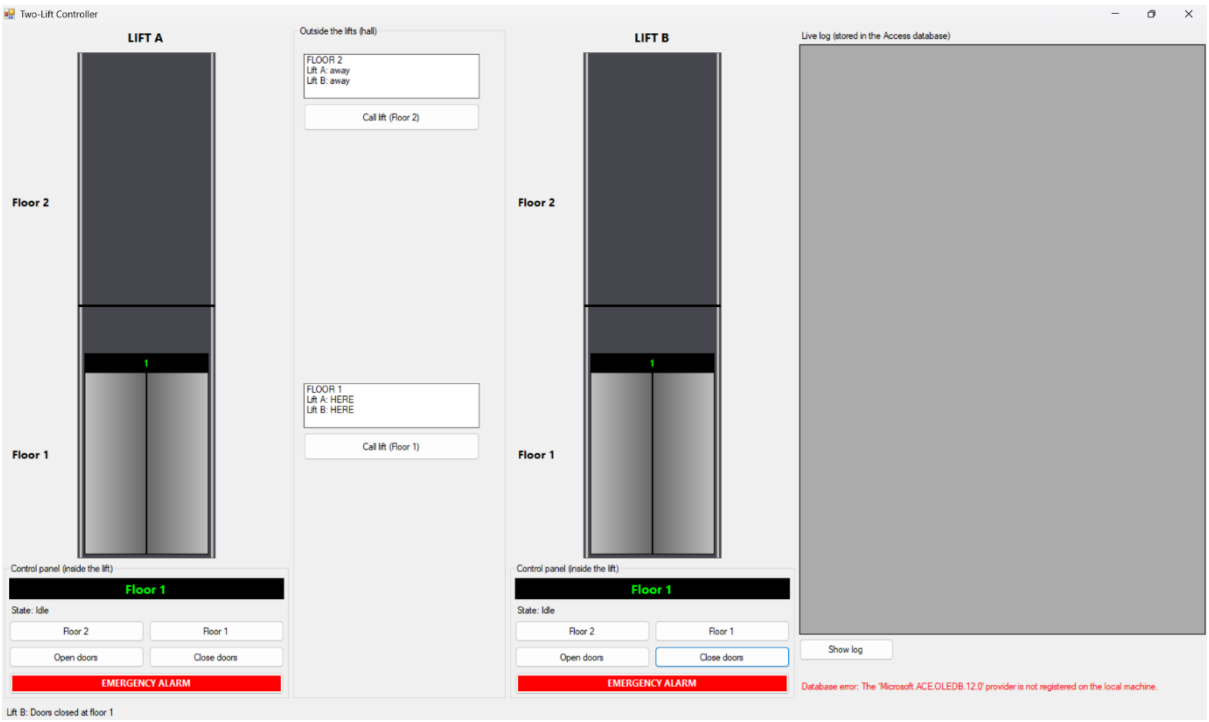
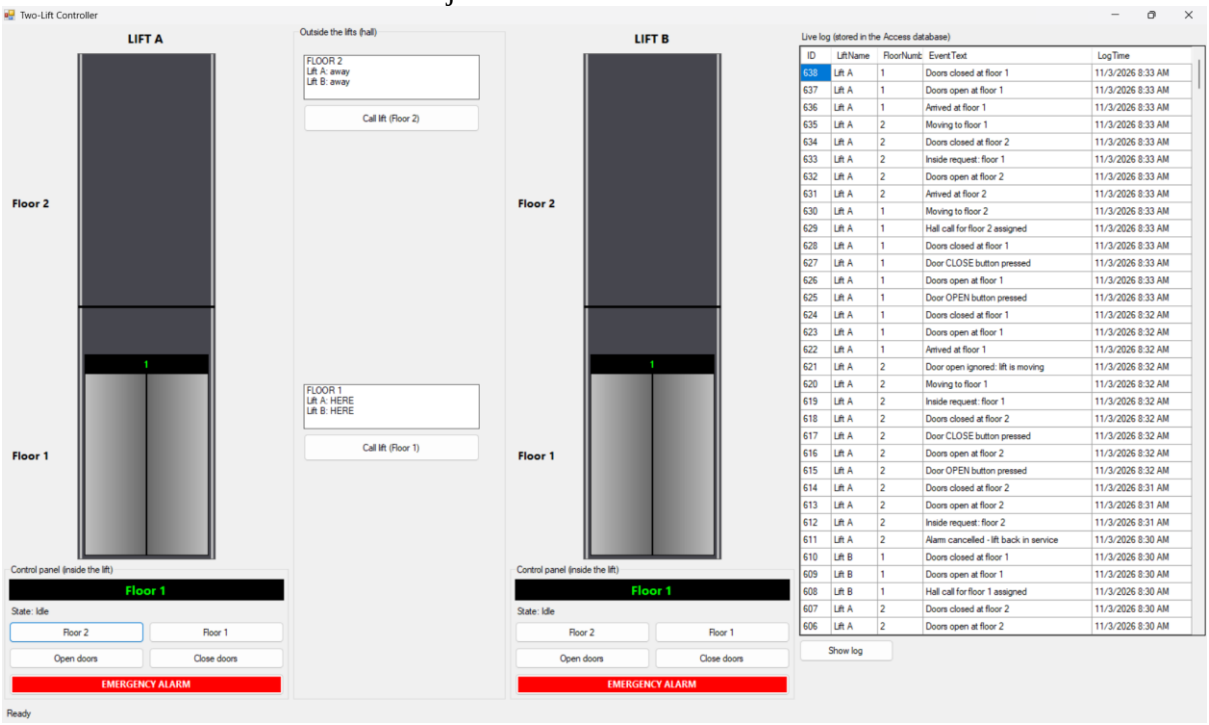
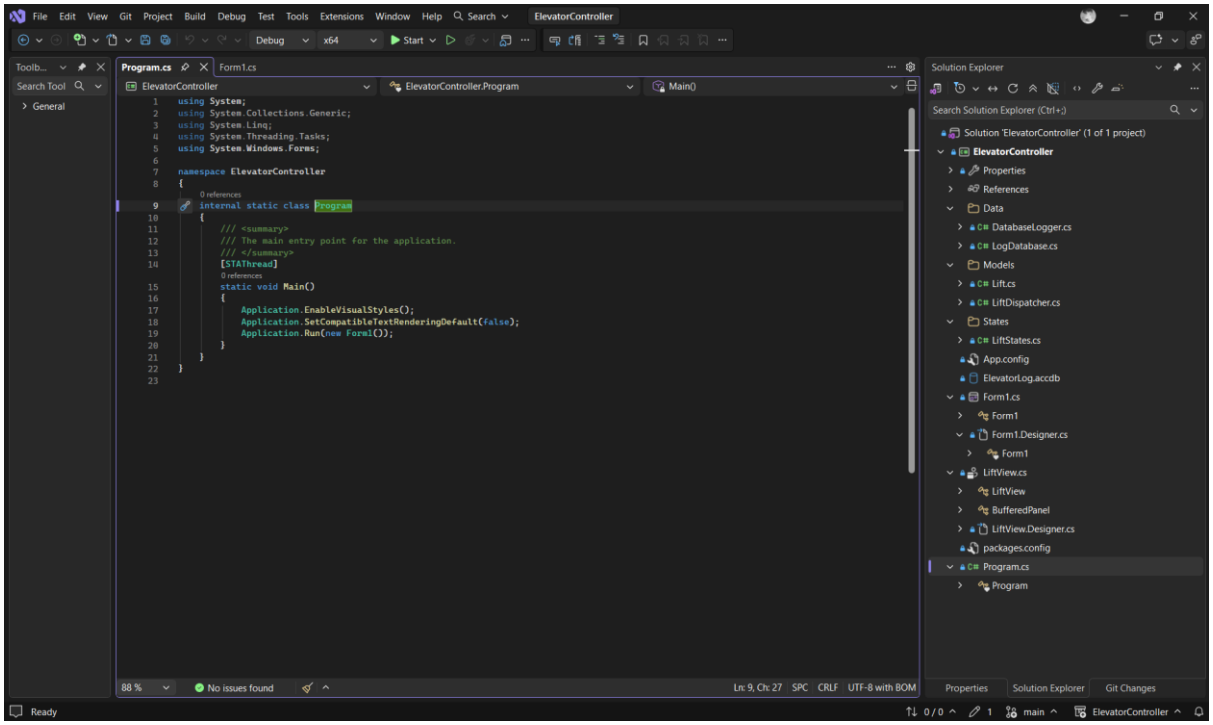


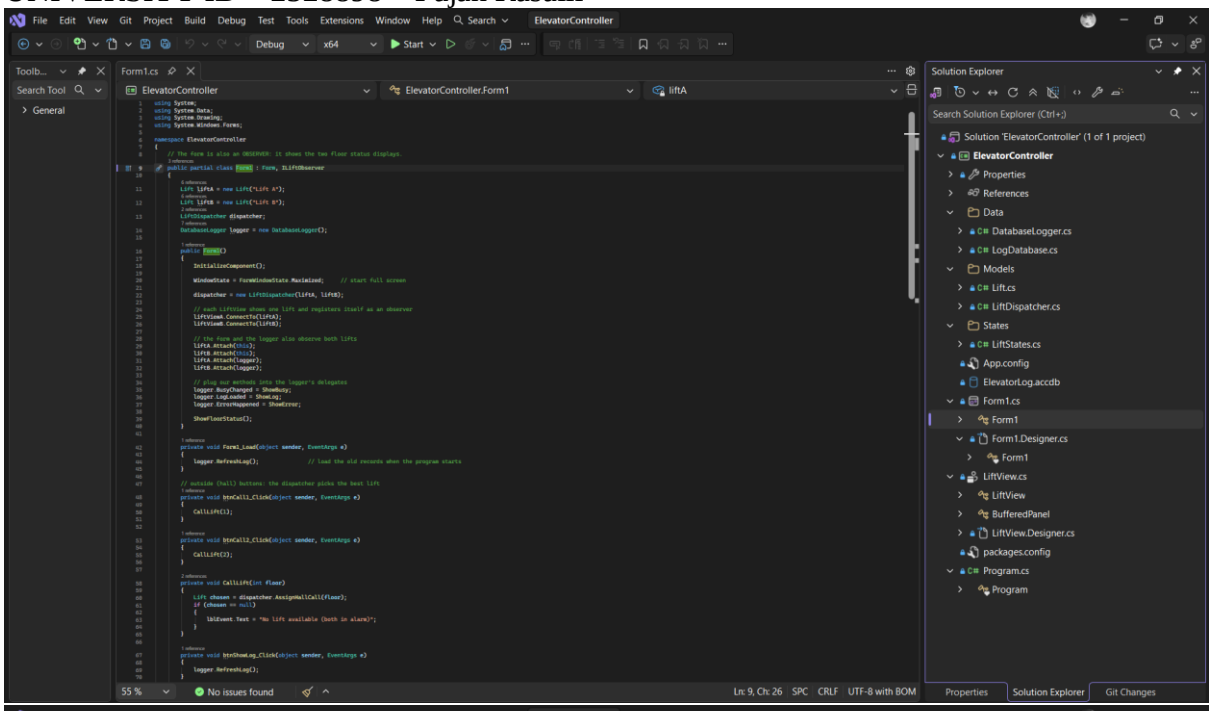
Figure 15 - Test 15: Extract the zip into a different folder and run it

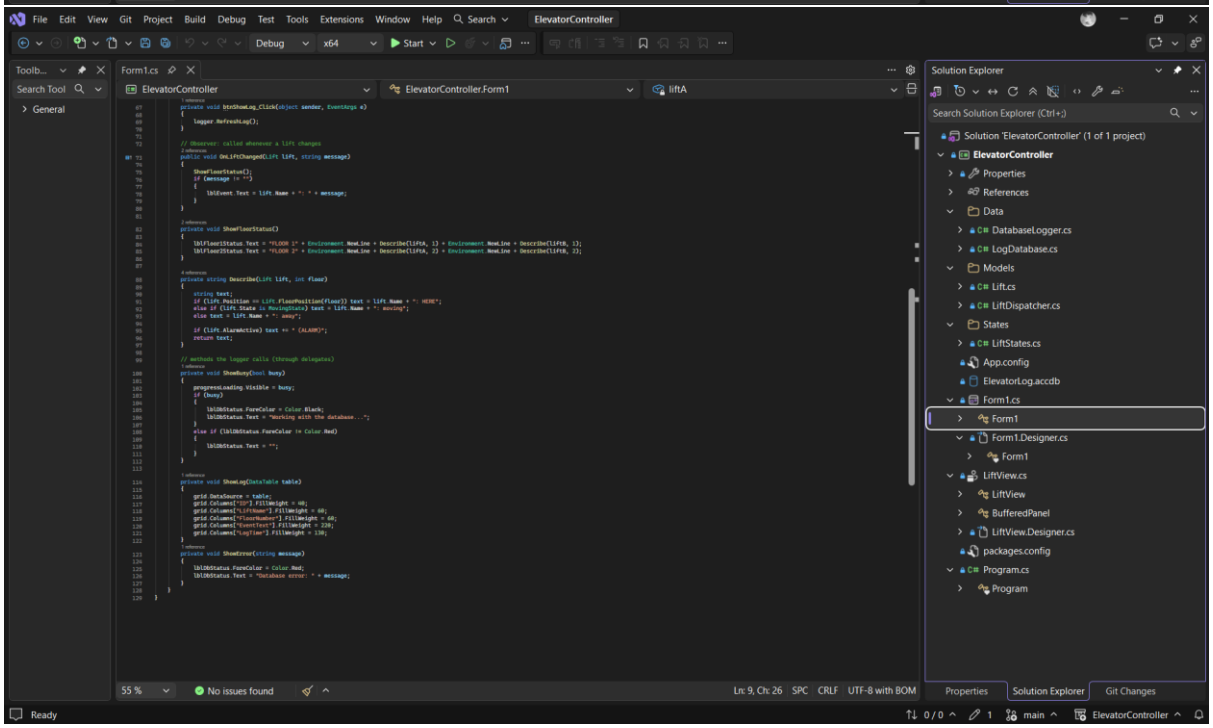


Appendix B: Source code

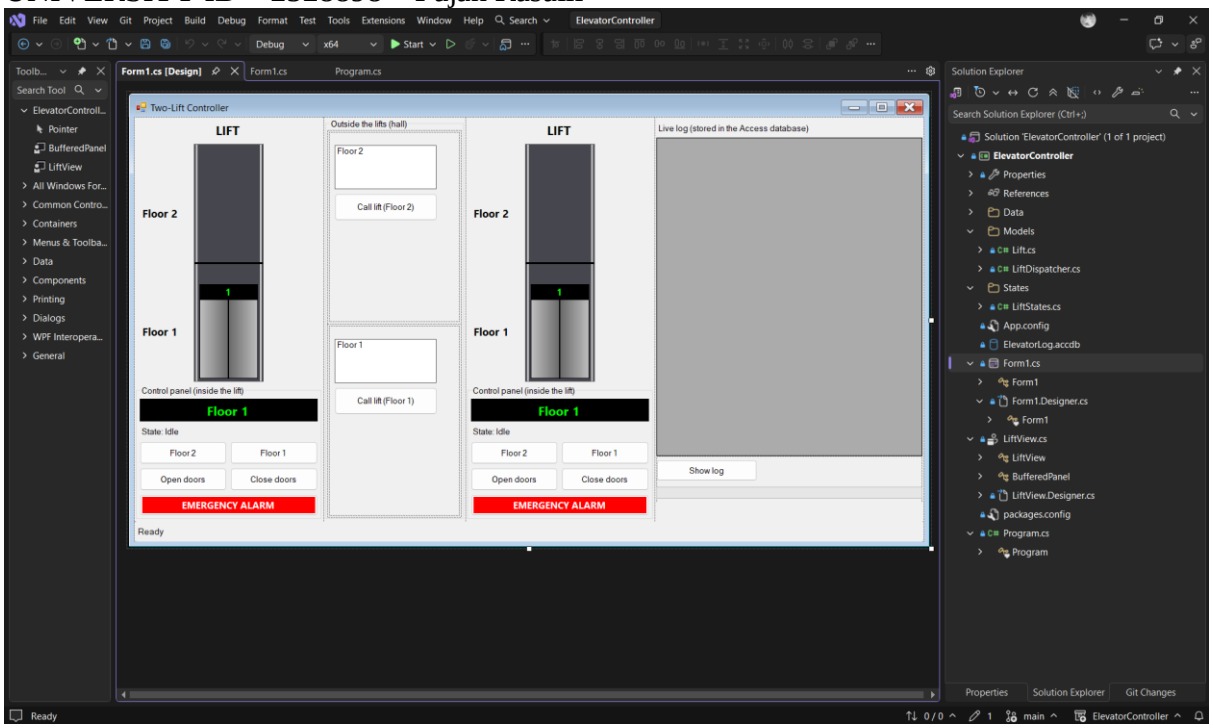


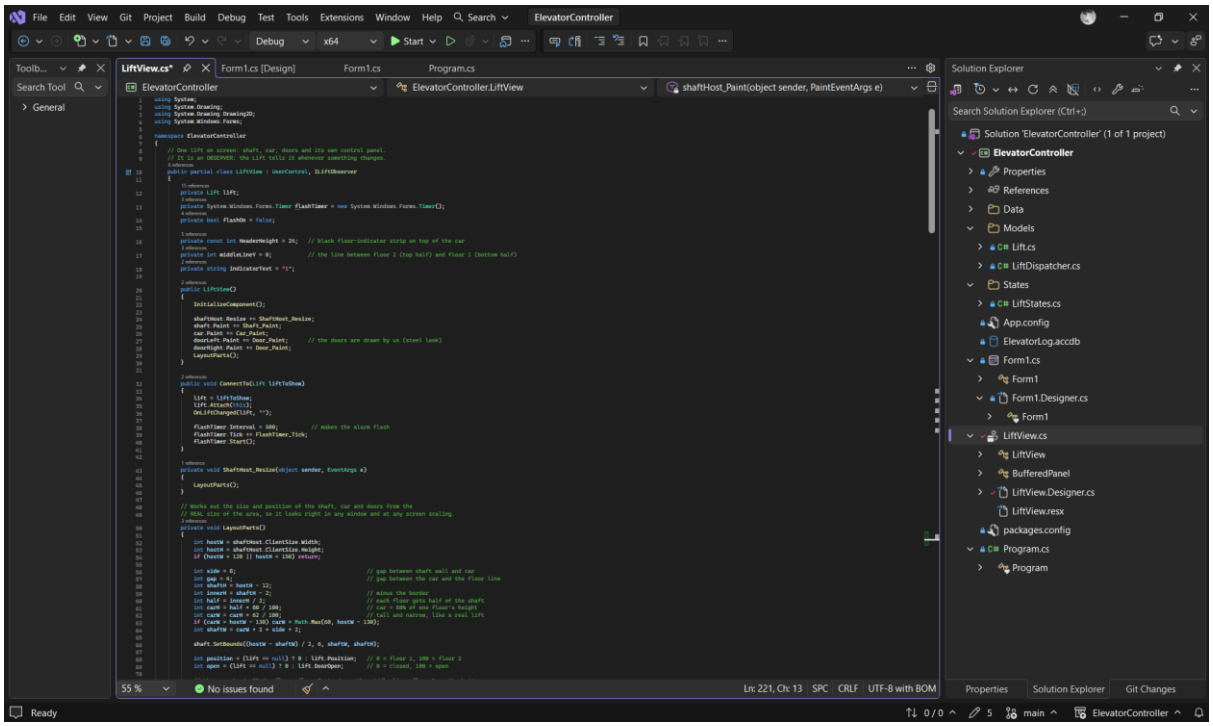
CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili



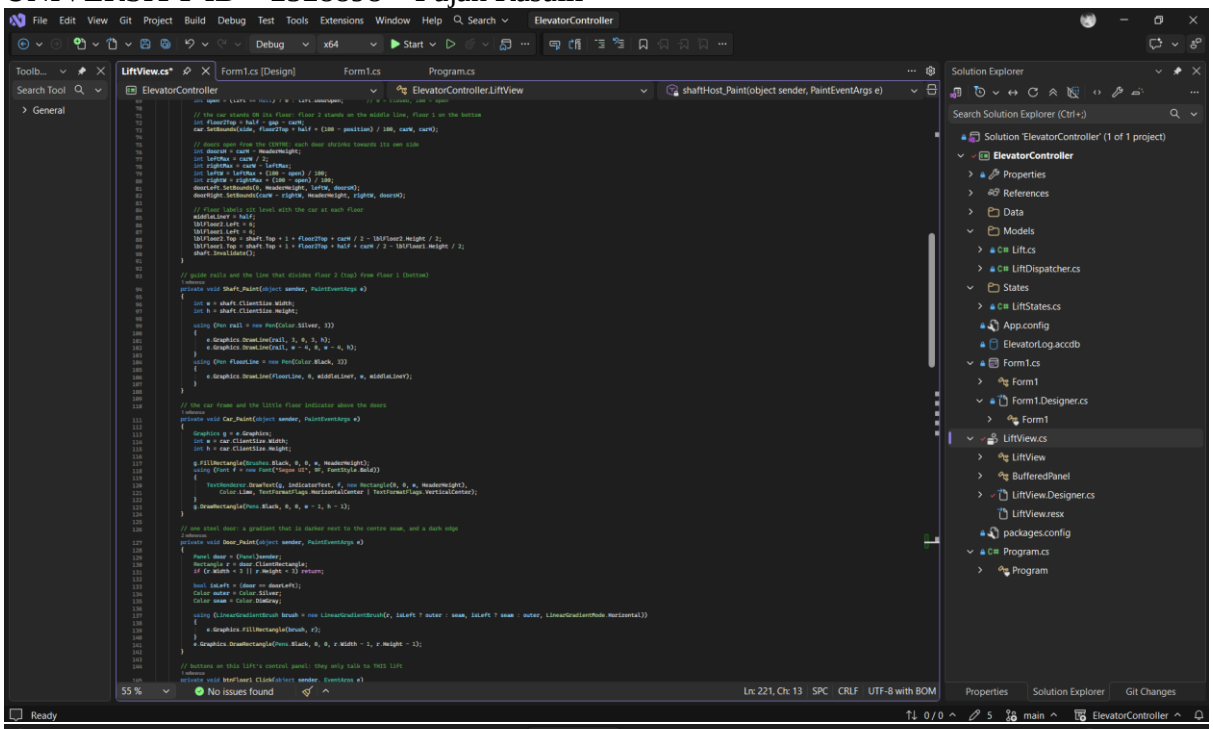


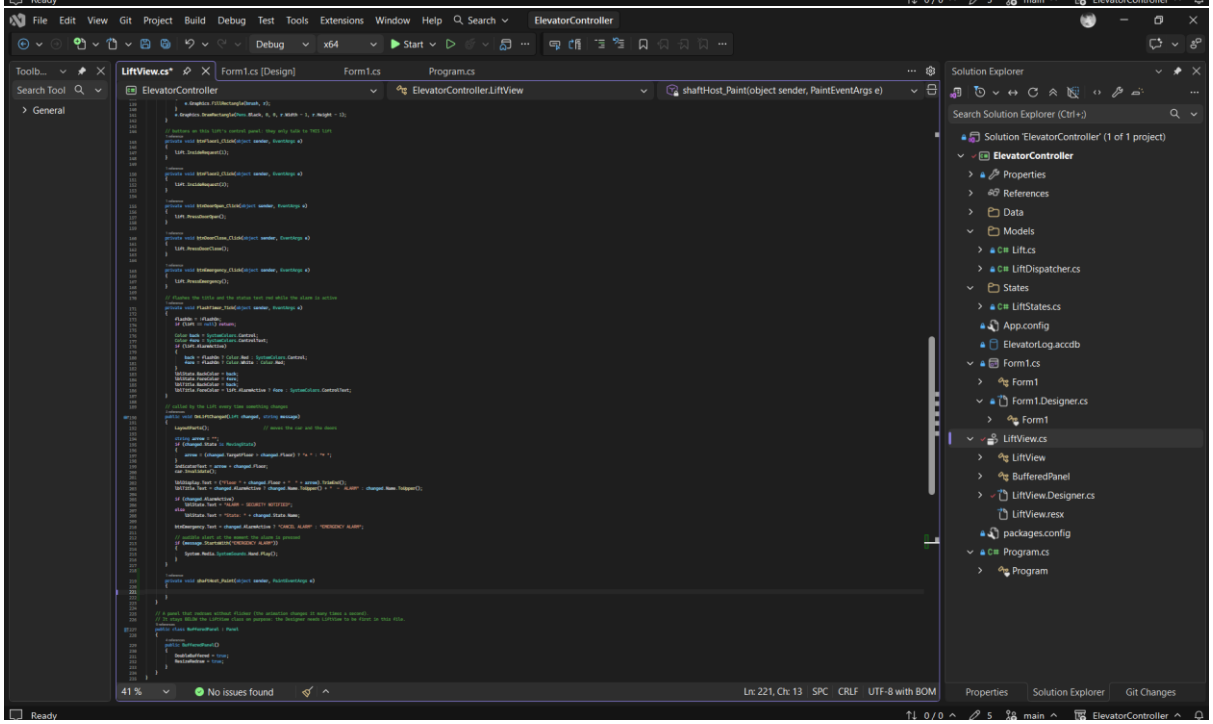
CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili



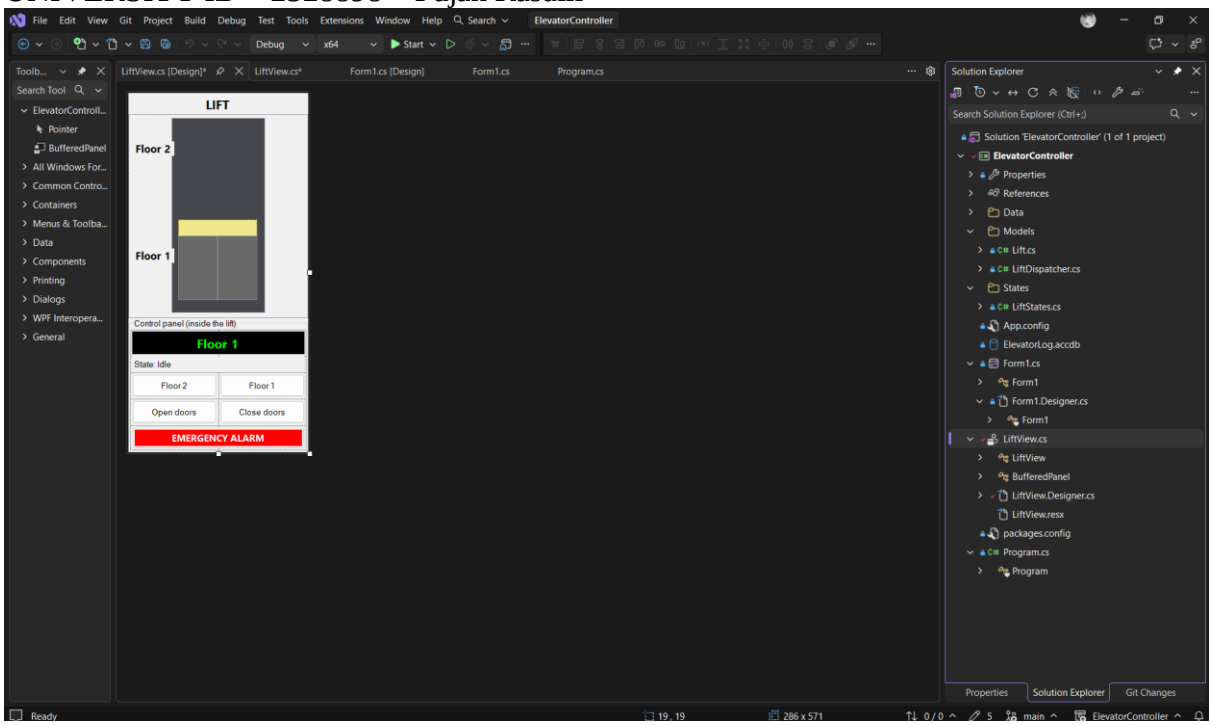


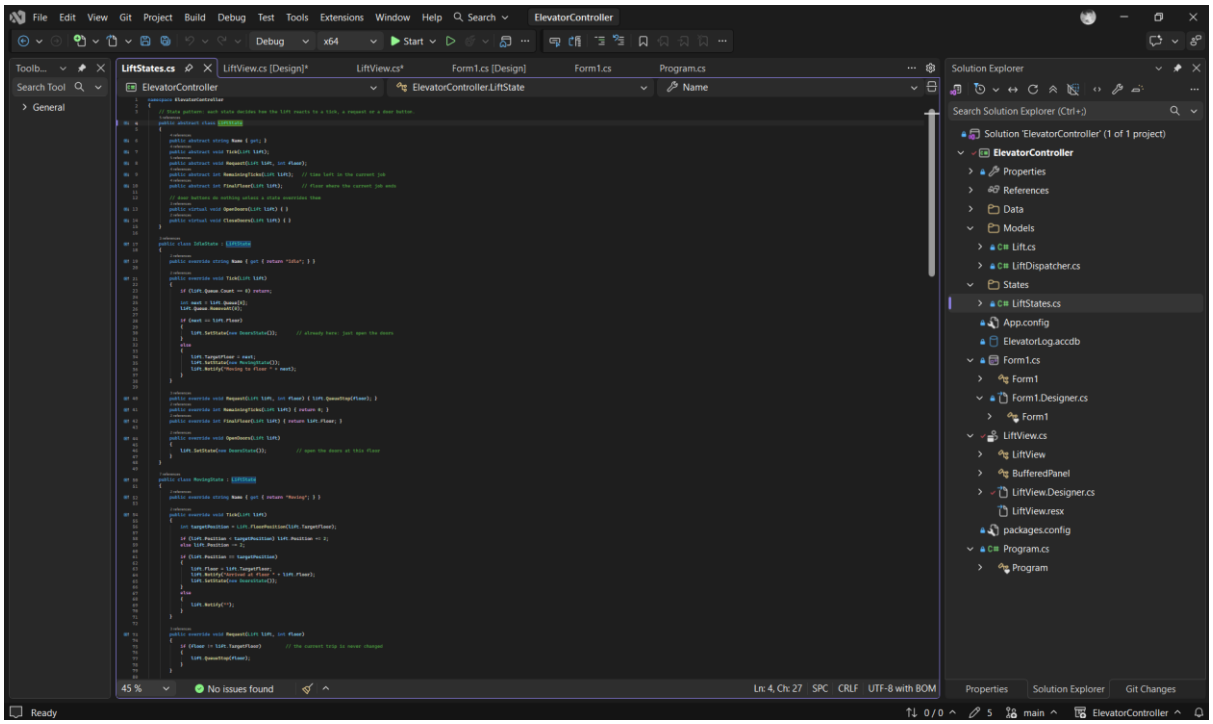
CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili



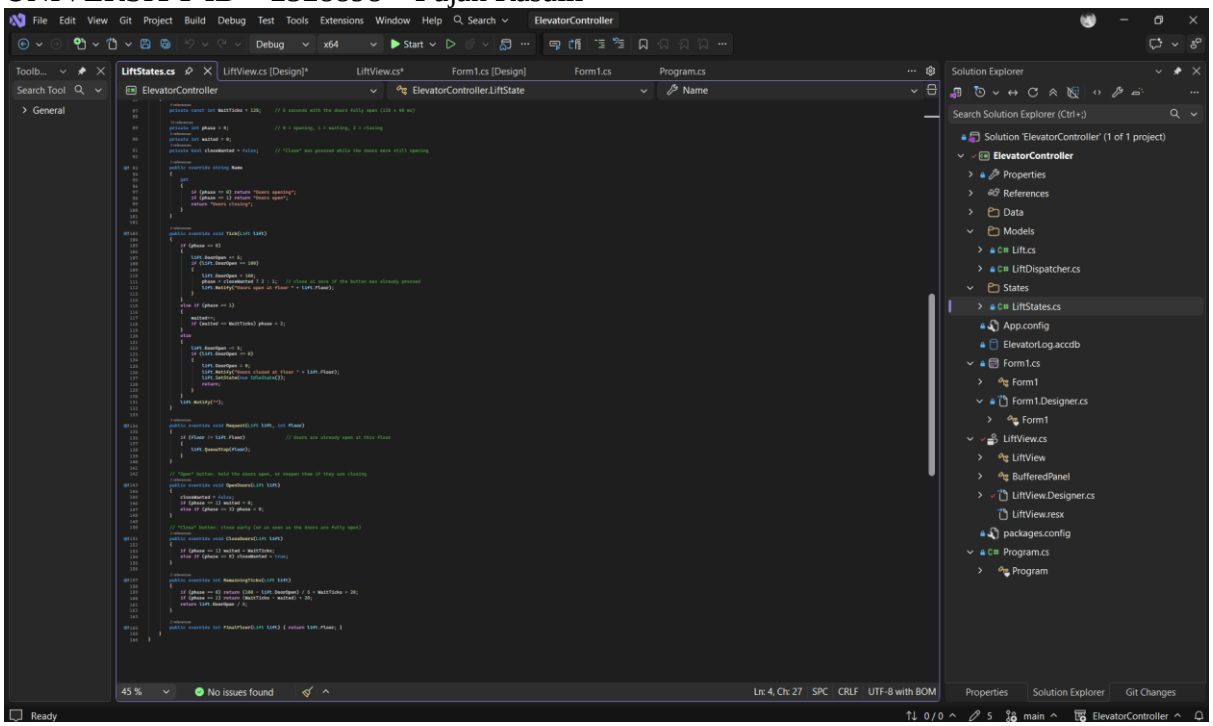


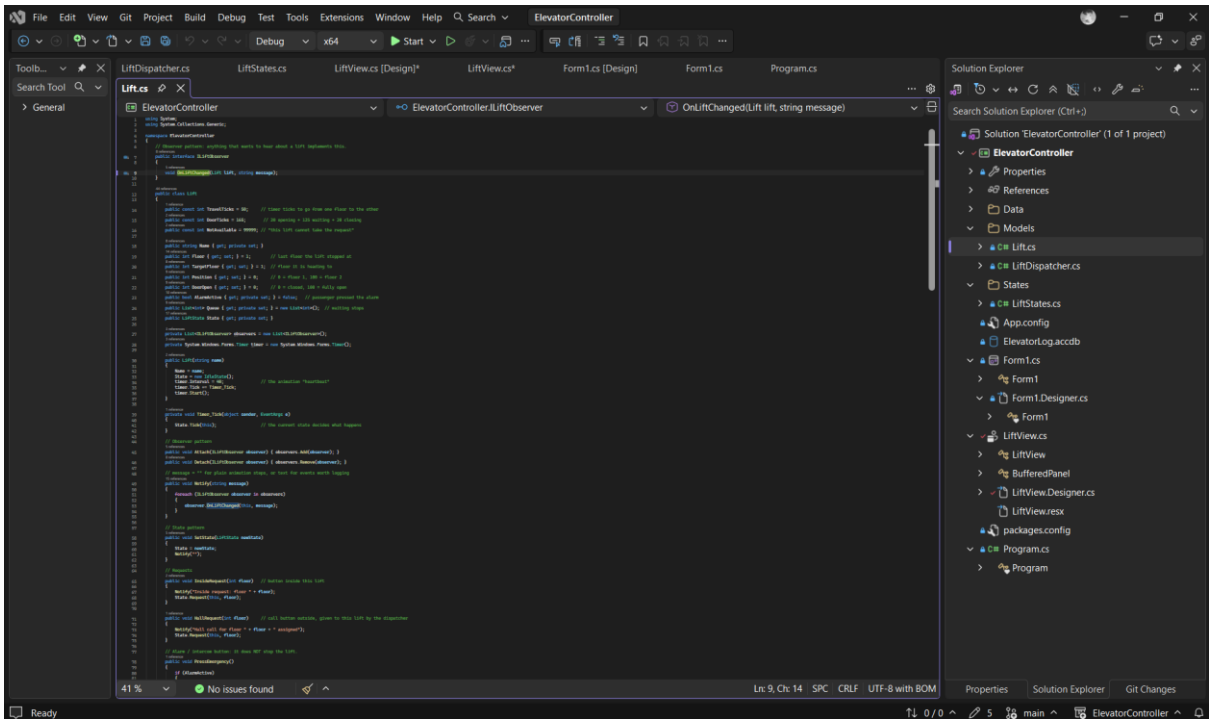
CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili



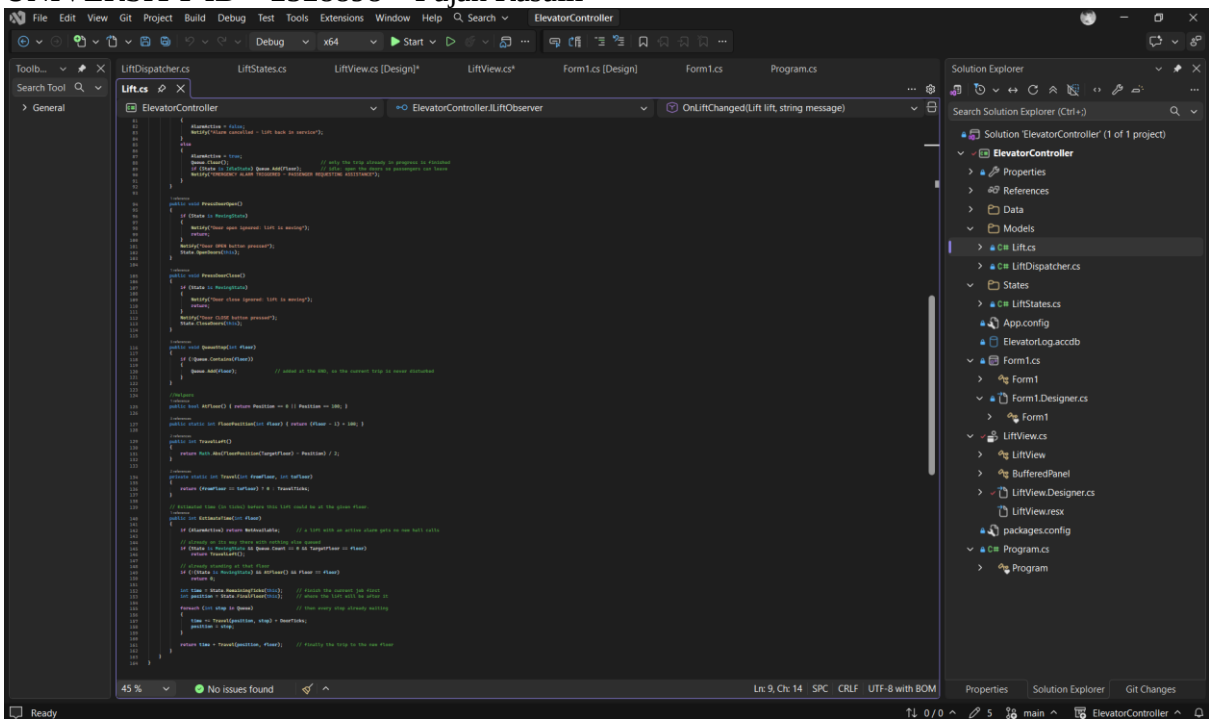


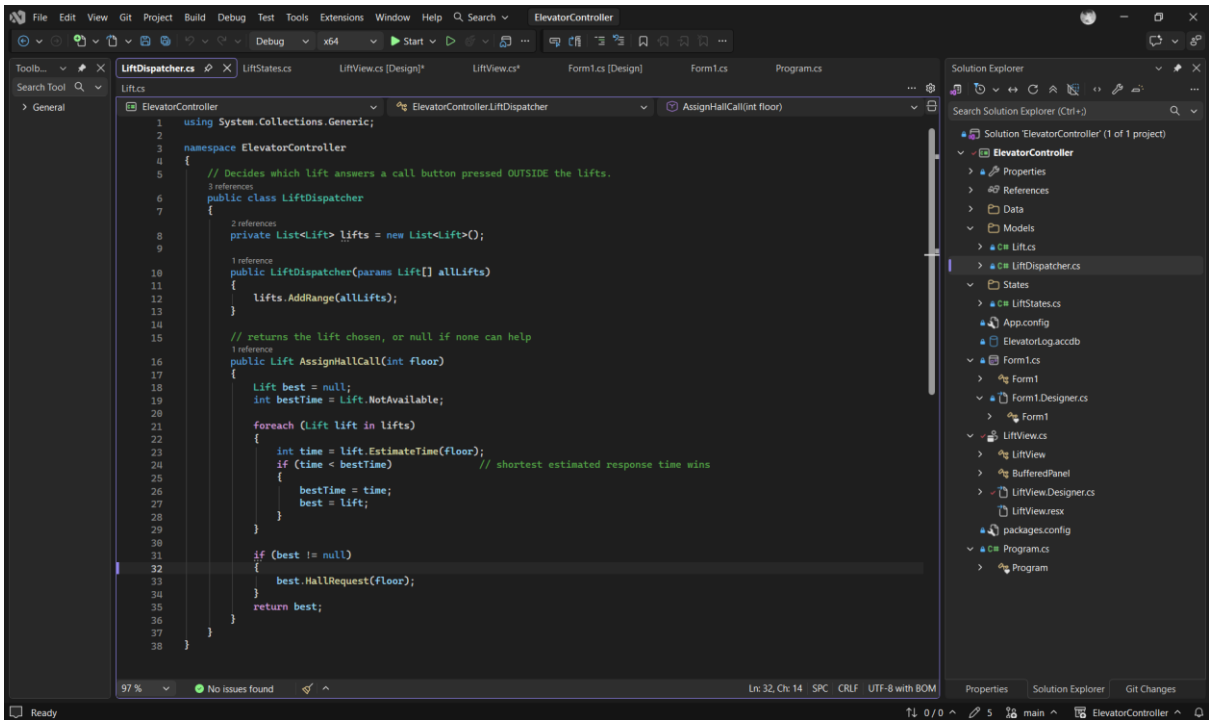
CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili

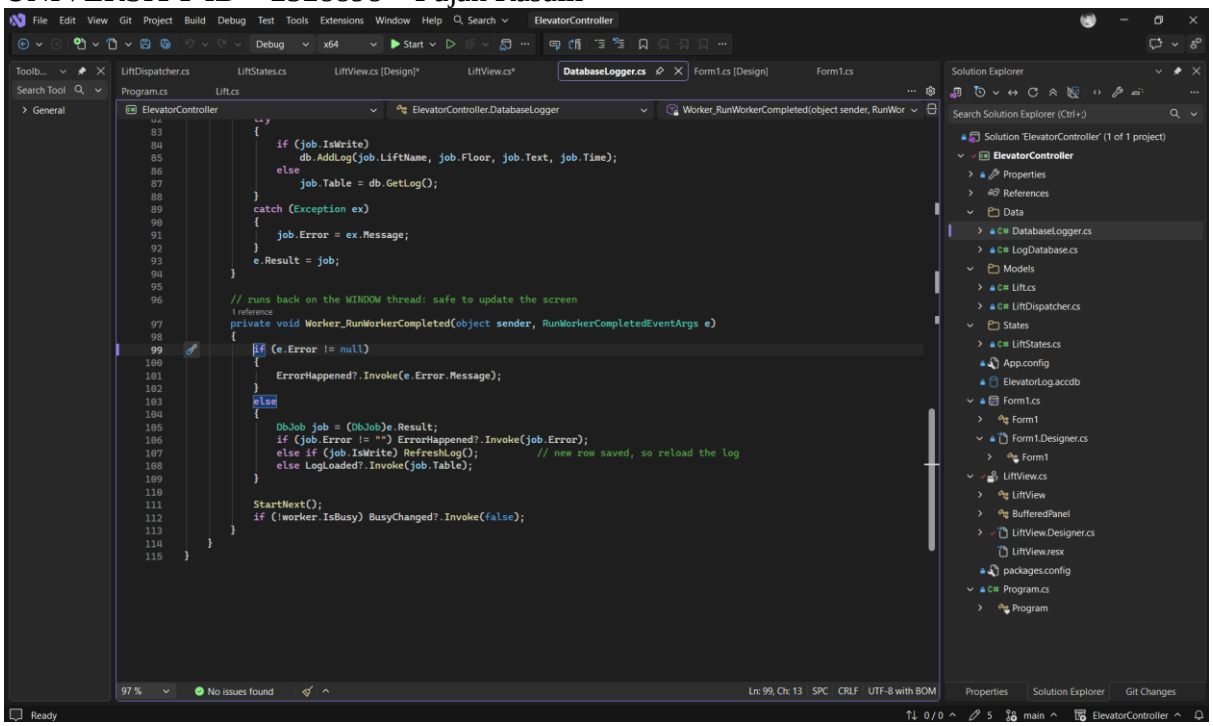




CIS116-2 – Desktop Application Development and Software Engineering
Assessment 1 –Control an Elevator - A C# Project
UNIVERSITY ID – 2528898 – Pujan Rasaili



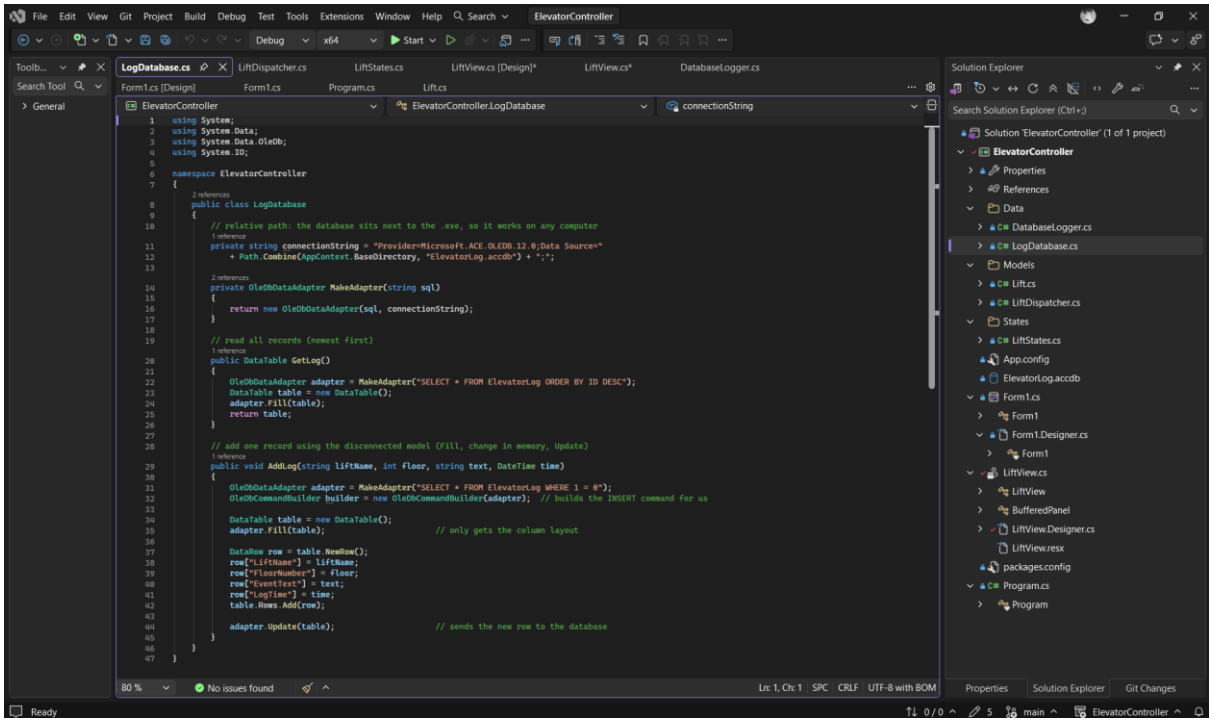




```
{
    if (job.IsWrite)
        db.AddLog(job.LiftName, job.Floor, job.Text, job.Time);
    else
        job.Table = db.GetLog();
}
catch (Exception ex)
{
    job.Error = ex.Message;
}
e.Result = job;
}

// runs back on the WINDOW thread: safe to update the screen
1 reference
private void Worker_RunWorkerCompleted(object sender, RunWorkerCompletedEventArgs e)
{
    if (e.Error != null)
    {
        ErrorHappened?.Invoke(e.Error.Message);
    }
    else
    {
        DbJob job = (DbJob)e.Result;
        if (job.Error != "") ErrorHappened?.Invoke(job.Error);
        else if (job.IsWrite) RefreshLog();          // new row saved, so reload the log
        else LogLoaded?.Invoke(job.Table);
    }

    StartNext();
    if (!worker.IsBusy) BusyChanged?.Invoke(false);
}
}
}
```



```
using System;
using System.Data;
using System.Data.OleDb;
using System.IO;

namespace ElevatorController
{
    2 references
    public class LogDatabase
    {
        // relative path: the database sits next to the .exe, so it works on any computer
        1 reference
        private string connectionString = "Provider=Microsoft.ACE.OLEDB.12.0;Data Source="
            + Path.Combine(AppContext.BaseDirectory, "ElevatorLog.accdb") + ";";

        2 references
        private OleDbDataAdapter MakeAdapter(string sql)
        {
            return new OleDbDataAdapter(sql, connectionString);
        }

        // read all records (newest first)
        1 reference
        public DataTable GetLog()
        {
            OleDbDataAdapter adapter = MakeAdapter("SELECT * FROM ElevatorLog ORDER BY ID DESC");
            DataTable table = new DataTable();
            adapter.Fill(table);
            return table;
        }

        // add one record using the disconnected model (Fill, change in memory, Update)
        1 reference
        public void AddLog(string liftName, int floor, string text, DateTime time)
        {
            OleDbDataAdapter adapter = MakeAdapter("SELECT * FROM ElevatorLog WHERE 1 = 0");
            OleDbCommandBuilder builder = new OleDbCommandBuilder(adapter);  // builds the INSERT command for us

            DataTable table = new DataTable();
            adapter.Fill(table);                    // only gets the column layout

            DataRow row = table.NewRow();
            row["LiftName"] = liftName;
            row["FloorNumber"] = floor;
            row["EventText"] = text;
            row["LogTime"] = time;
            table.Rows.Add(row);

            adapter.Update(table);                  // sends the new row to the database
        }
    }
}
```

Appendix C: Source code repository

https://github.com/pujanrasaili/ElevatorController-CIS0116-2